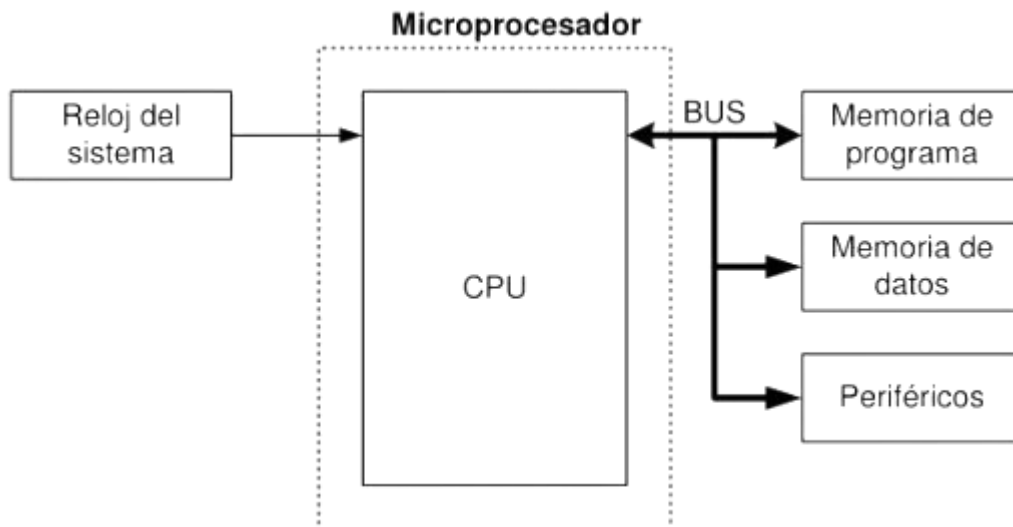


DIGITAL II TEORICO 1

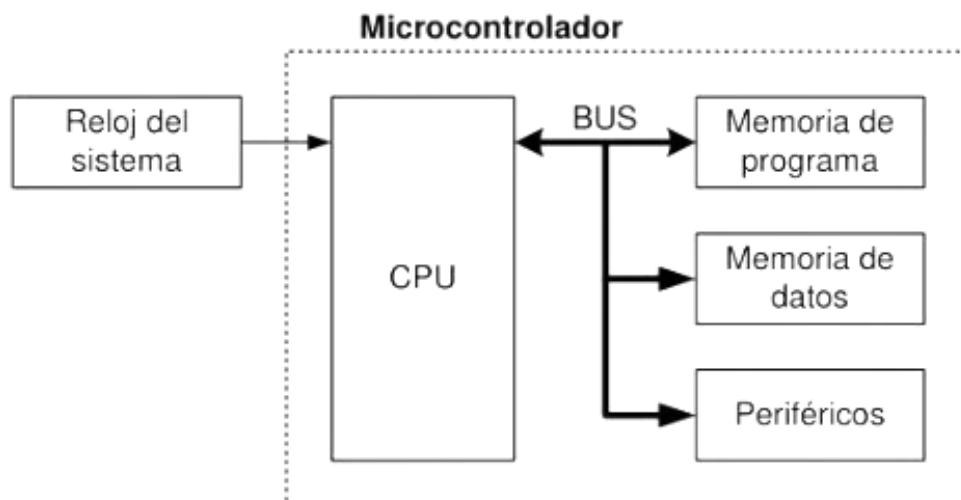
MICROPROCESADOR

- Circuito integrado que contiene la CPU
- Precisa de otros circuitos integrados para funcionar
- Sistema escalable con gran potencia computacional
- Sistema operativo
- Es un **gran circuito secuencial**, es un circuito que gracias a la ayuda de un reloj va cambiando de estado de acuerdo a variables de control que suelen ser señales.
- Se comunica principalmente con 3 grandes periféricos: la memoria de programa, la memoria de datos y los periféricos que son puertos de entrada y salidas



MICROCONTROLADOR

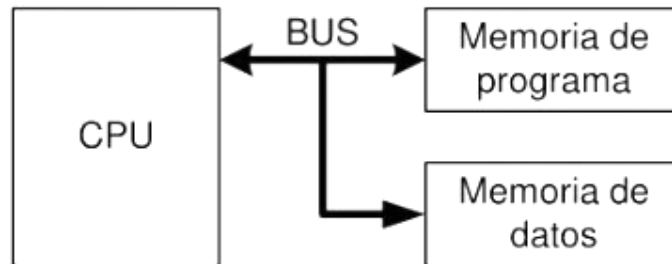
- A diferencia del microprocesador, tiene integrado la CPU, la memoria de programa, la memoria de datos y los periféricos.
- Dedicado a una sola tarea, ya que es programable. Uno lo programa y lo deja funcionando.
- Es un sistema completo y está preparado para funcionar con lo que tiene, no es escalable (no están destinados a ser ampliados o mejorados en términos de rendimiento o capacidad como se haría con un sistema de microprocesador).



ARQUITECTURAS DEL PROCESADOR

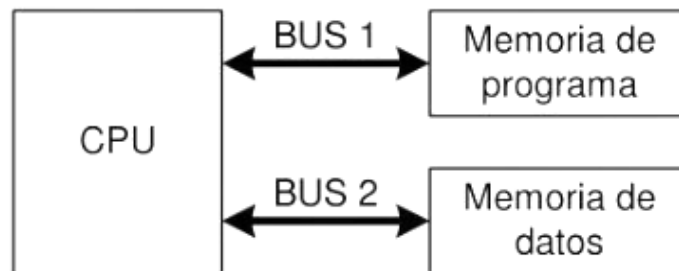
- **Von Neumann:**

- Memoria común para datos e instrucciones.
- Arquitectura clásica, la mayoría de los microcontroladores utilizaban esta.
- Asociada a CPU CISC



- **Harvard**

- Memoria de datos y memoria de programa son distintas. Esto nos permite mejorar la ejecución del programa.
- Se incrementa la capacidad de direccionamiento.
- Mejora el rendimiento y la eficiencia del microcontrolador permitiendo accesos simultáneos y optimizados a las memorias de programa y datos
- Asociada a CPU RISC



Unidades de procesamiento central CPU:

- **CISC**

- Complete Instruction Set Computer
- Muchas instrucciones con muchos modos de direccionamiento
- Número de ciclos por instrucción variable.

- **RISC**

- Reduced Instruction Set Computer
- Pocas instrucciones muy sencillas
- Número de ciclos por instrucción constante

Memoria de Programa: Memorias en donde la información no se borra al apagar el sistema

- **ROM:** Read Only Memory
- **OTP:** One Time Programmable
- **EPROM:** Erasable Programmable Read Only Memory
- **EEPROM:** Electrical Erasable Programmable Read Only Memory
- **FLASH:** Igual que EEPROM pero con menos consumo y costo, mayor velocidad y densidad de integración.

Memoria de dato:

- **RAM:Random Access Memory.** Si apago el sistema, la información se borra.
 - **SPRAM:** Static RAM.
 - **DRAM:** Dynamic RAM.
 - **SDRAM:** Synchronous Dynamic Ram.

Dentro del micro vamos a tener una parte de memoria EPROM y una parte de memoria RAM

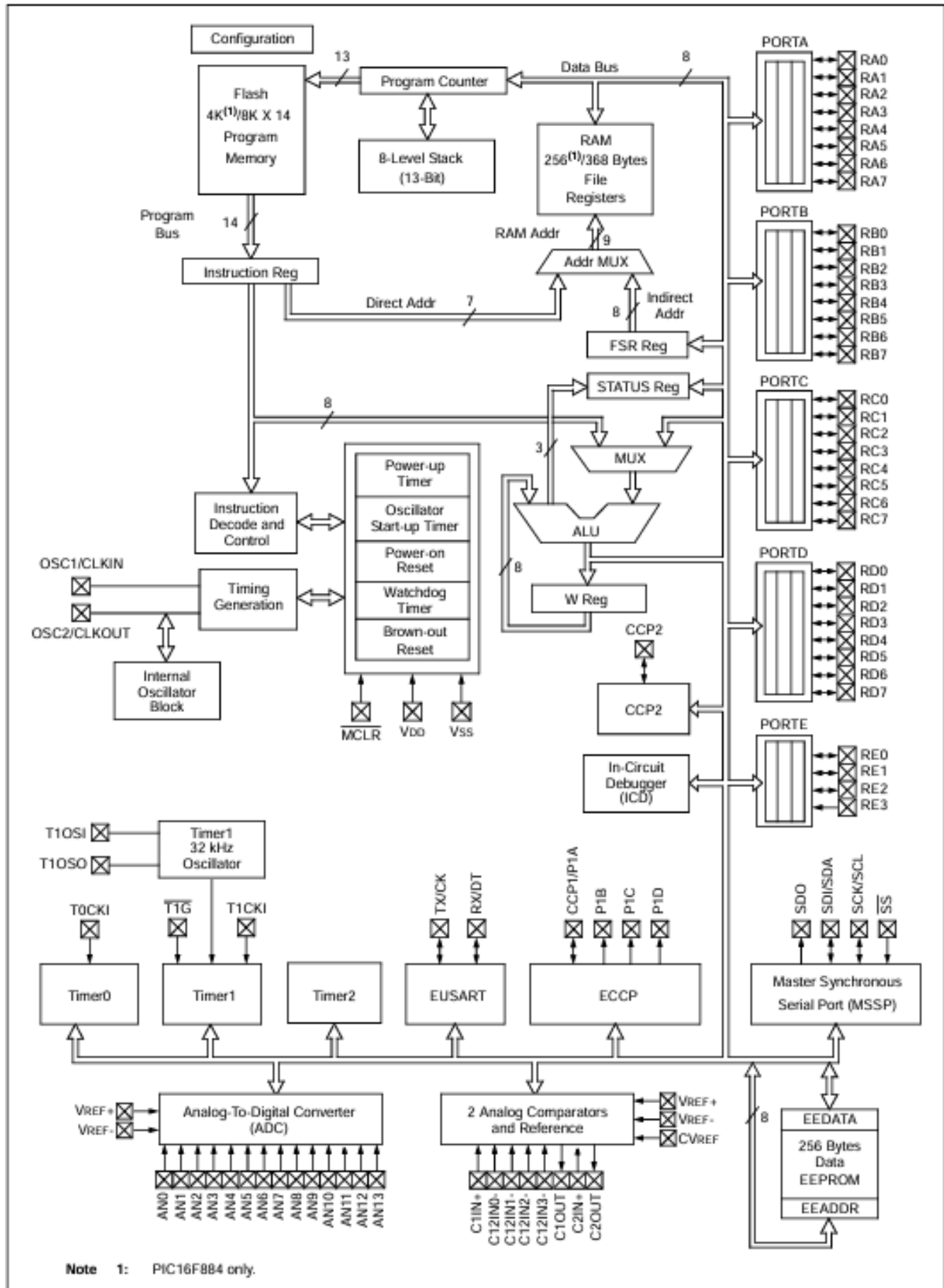
PERIFÉRICOS

- **Perro guardián (watchdog)**, temporizador especial que reinicia el sistema si detecta que el mismo se ha quedado en un bucle o ha fallado.
- **Detector de caída de la alimentación (brownout)**, sistema que ante una caída abrupta de tensión, guarda la información importante y apaga el micro de manera segura.
- **Temporizadores (timers)**, miden intervalos de tiempo. Pueden generar interrupciones, medir la duración de eventos o generar señales temporales.
- **Modulador de anchura de pulsos (PWM)**
- **UART**, sistema de transmisión serie.
- **Convertidor A/D**
- **Puertos de entrada/salida**, lugares a través de los cuales un microcontrolador puede recibir datos(entrada) o enviarlos (salida).

REGISTROS: Lugares donde se guarda información, normalmente agrupado de 8 bits. Es un conjunto de 8 flip flops

Los más comunes son:

- PC, Program Counter
- Working Register

FIGURE 1-2: PIC16F884/PIC16F887 BLOCK DIAGRAM

UNIDAD ARITMÉTICA LÓGICA - ALU

- Tiene dos entradas, en una entra siempre el registro de trabajo W, y en la otra pueden entrar distintas cosas de acuerdo a lo que el micro quiera realizar.
- Funciona con 8 bits

REGISTRO STATUS

- Guarda el contenido de algún tipo de bandera de estado que se va moviendo durante el transcurso del programa (C,DG,Z).

FSR-File Selection Register

- Permite direccionamiento del tipo indirecto.

STACK

- Es una pila que permite almacenar hasta 8 direcciones de retorno de subrutina o interrupciones.
- No se puede acceder de manera directa, solo a través de instrucciones del tipo CALL, RETURN y RETFIE.
- Es un tipo de memoria LIFO (last in, first out)

¿Qué es el mapeo?

Ubicar dentro del mapa de memoria a un dispositivo.

Mapeo directo→ A partir del mapa busco el circuito

Mapeo inverso→ A partir del circuito busco el mapa

Mapeo absoluto→ Utilizo todos los pines de la memoria

Mapeo relativo→ Dejo uno o más pines sin conectar. Esto me va a producir reflejos, si tengo n pines sin utilizar la memoria se me va a reflejar 2^n veces.

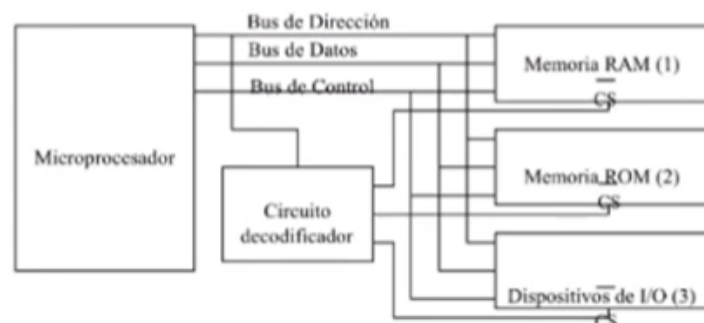
[DIGITAL II TEORICO 2 15032024](#)

MAPEO DE MEMORIA

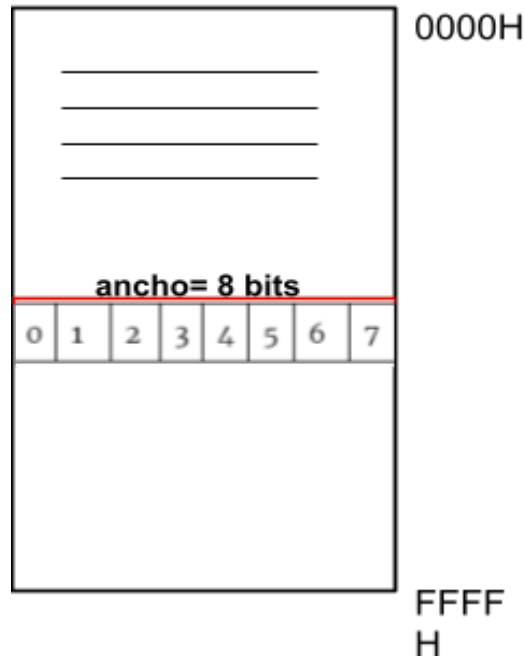
- **Bus de direcciones:** Nos indica a donde se van a guardar los datos que enviamos.
- **Bus de datos:** Para enviar los datos
- **Bus de control:** Con el indicamos que memoria se va a escribir o leer.

El **circuito decodificador** nos permite proveer a las memorias y los dispositivos de I/O de una zona exclusiva del mapa de memoria del micro. Cualquier dispositivo que se conecte al microprocesador, debe disponer de una bus de direcciones igual o menor al que provee el micro, de esta manera podemos, para un dispositivo dado, distinguir dos tipos de líneas de direccionamiento.

- **Líneas de direccionamiento externo**, encargadas de diferenciar el dispositivo del resto (dispositivo 1, 2 o 3), de esta forma puedo tener varios en un sistema de buses y acceder a uno en especial por medio de estas líneas y un circuito decodificador.
- **Líneas de direccionamiento interno**, son las que se conectan directamente al dispositivo.



De acuerdo a la cantidad de direcciones que puede tener un micro tengo un espacio de memoria, el cual puedo direccionar. Este espacio es donde el micro puede posicionar datos. El tamaño máximo que puede trabajar el micro es de 64 KB posiciones.

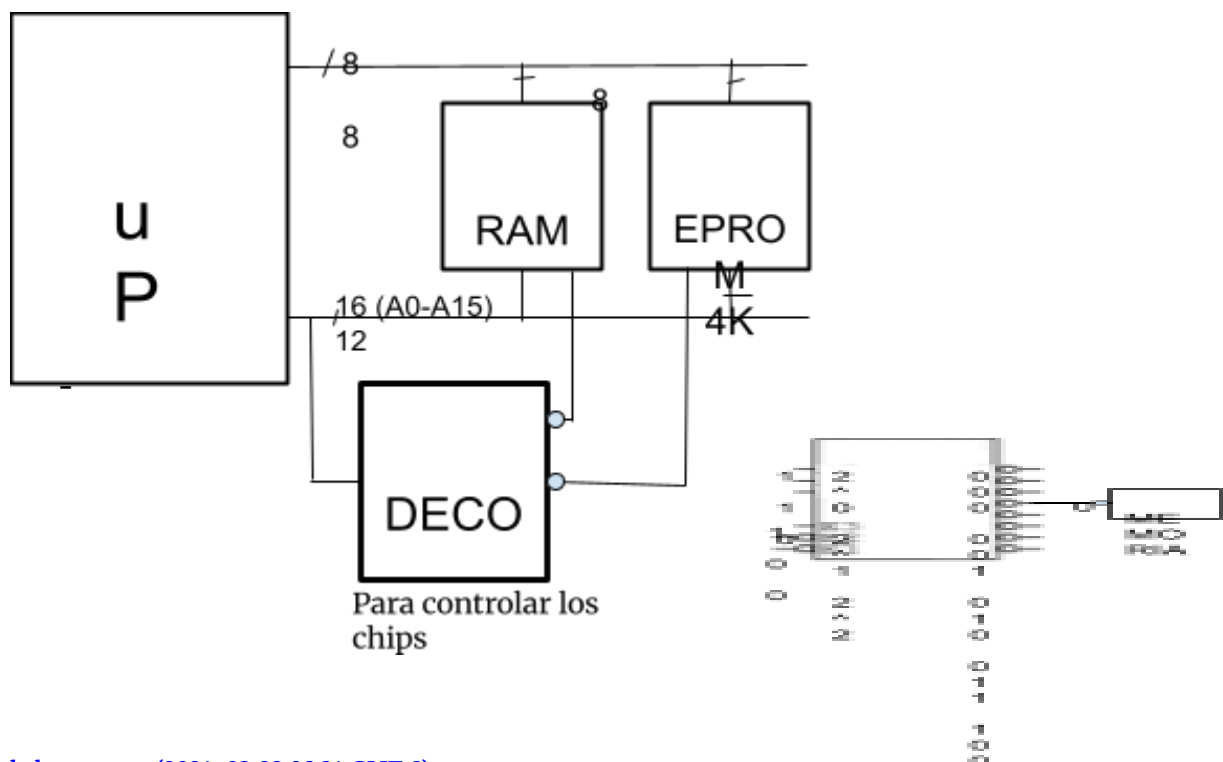


Podemos direccionar desde el 0000H hasta FFFFH. Esto no significa que yo tengo todo esto como memoria, yo tengo la posibilidad, dependiendo de la cantidad de memoria que tenga y en el lugar que las ponga, es como yo voy a configurar el sistema de memorias.

El ancho es la palabra/dato que voy a escribir, luego 0000H-FFFFH son las direcciones.

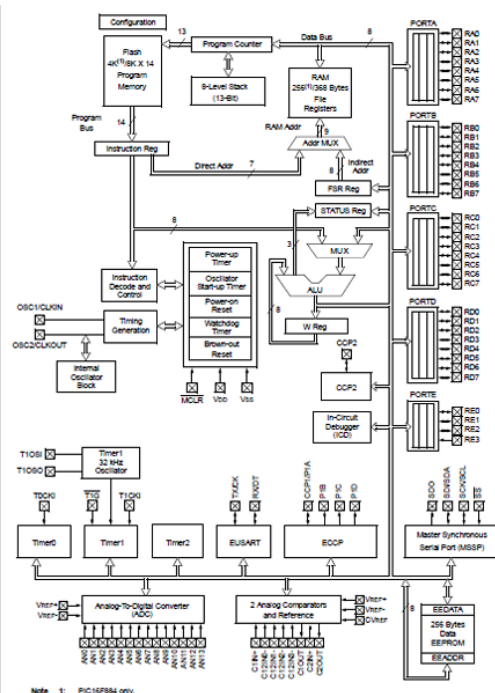
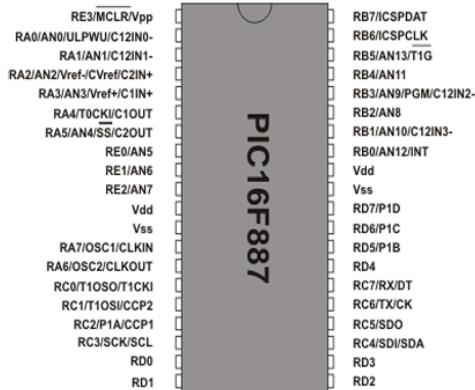
Vectores de dirección importantes: 0000H y 0004H

Entonces, vamos a trabajar con 1 byte de palabra/dato y 64Kb de direccionamiento



[hcb-zrya-ysg \(2024-03-22 08:34 GMT-3\)](#)

En la imagen se observa el esquema físico y el circuital del PIC16F887



RECORDAR

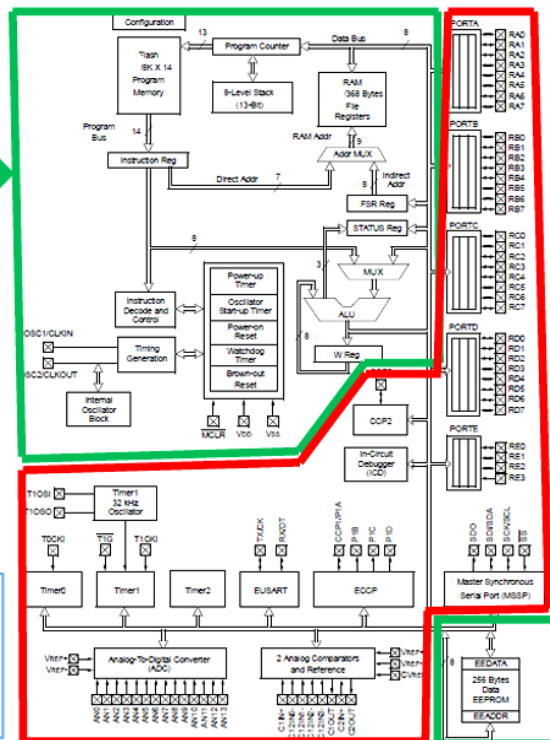
- La diferencia entre un **microcontrolador** y un **microprocesador**, es que el primero tiene integradas otras cosas además del CPU.
- Este micro tiene **arquitectura Harvard**, memoria de programa separa de memoria de datos

NUCLEO CPU + MEMORIAS

ALU, sincronización y control, (puertos E/S, ADC, PC, clock, etc.)

ARQUITECTURA

Para analizar la arquitectura del PIC16F887 comenzaremos analizando el núcleo y luego cada uno de los módulos periféricos.

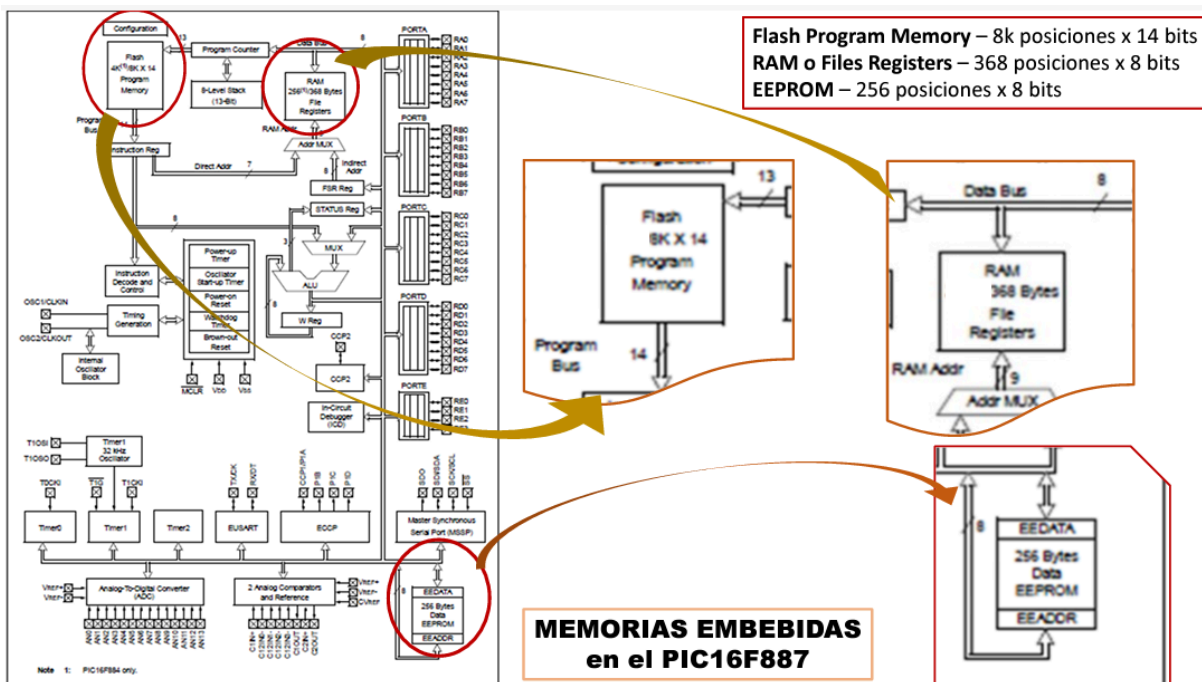


PERIFERICOS

(puertos E/S, ADC, temporizadores, comparadores, PWM, Transmisión serie)

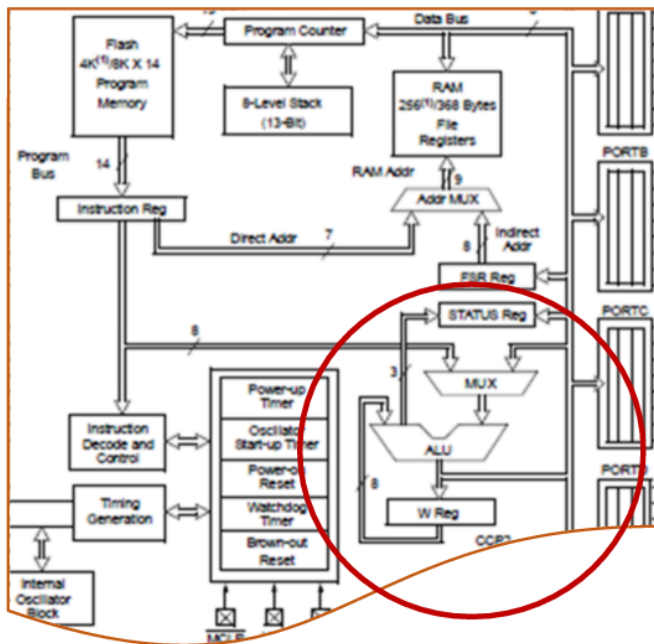
Cuenta con 3 tipos de memorias:

- **Memoria de programa tipo FLASH de tipo EEPROM:** permite ser borrada eléctricamente y tiene un bloque de 8K para la memoria de programa. Es más barata que una EEPROM normal.
 - **Registro PC** (program counter) de 13 bits, permite direccionar todo el sistema de memoria interna del micro (apunta a la dirección que estoy ejecutando)
 - **Registro Stack** (8 niveles, puede almacenar hasta 8 valores, si almaceno uno más empiezo a perder datos) es una memoria destinada para almacenar direcciones de retorno de subrutinas e interrupciones. Memoria LIFO.
 - **Bus de dato** de 8 bit
- **RAM estática** (formada por filp flop), destinada a almacenar registros temporales, ya que es una memoria volátil, es decir, cuando apago el sistema ésta se borra.
 - **Registro STATUS** almacena el contenido de algún tipo de banderas de estado que se van moviendo durante el transcurso del programa.
- **EEPROM** (256 Bytes), se utiliza para almacenar cosas que no quiero que sean borradas, por ejemplo programaciones iniciales del micro.

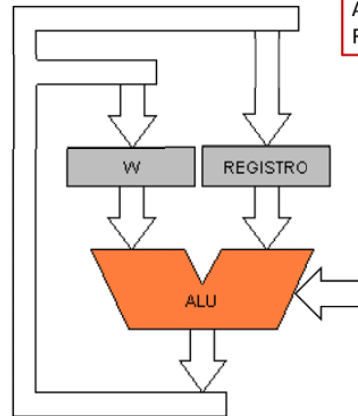


El micro cuenta con un par de pines para la conexión de un cristal externo (oscilador a cristal muy preciso). También cuenta con una entrada para un oscilador interno.

Cada ciclo de instrucción requiere 4 oscilaciones de cristal (4 ciclos de reloj) → el micro por dentro trabaja a una frecuencia 4 veces menor a la del cristal.



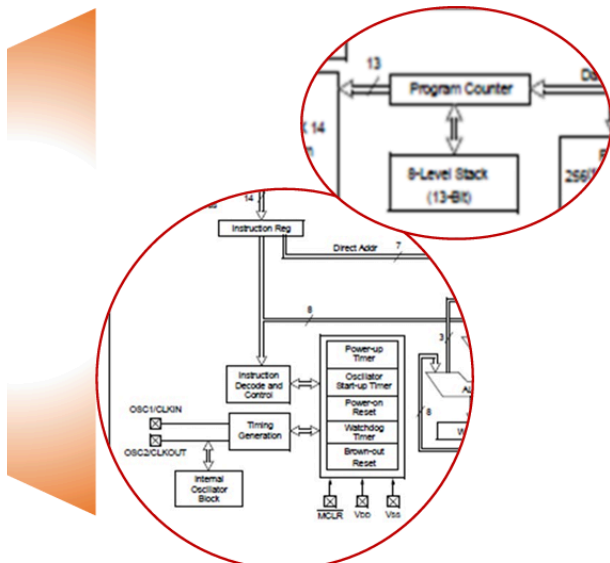
**UNIDAD ARITMETICA LOGICA – ALU
en el PIC16F887**



ALU de 8 bits
Registro de trabajo: W

Todas o casi todas las operaciones que realizamos dentro del micro se realizan con ayuda o a través del registro W. Casi todos los resultados van a terminar allí.

El resultado de estas operaciones puede tener diversas características (puede ser 0, puede generar carry), y éstas son enviadas a través de un bus auxiliar al registro STATUS.



**CIRCUITOS DE SINCRONIZACION Y CONTROL
en el PIC16F887**

Todas las instrucciones se ejecutan en 4 ciclos del oscilador excepto las de salto (goto, call, etc.)

Estos circuitos son muy importantes, porque el micro es un gran secuencial, con lo cual necesitamos sincronizarlo de alguna manera.

En estos bloques vemos:

- Program Counter, permite direccionar las memorias

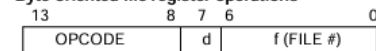
- Stack

- Decodificación y control de instrucciones:

Cada Instrucción entra en una palabra de 14 bits. Parte de esa palabra está destinada al OPCODE (código de operación), éste le dice al micro cual es la operación que va a ejecutar

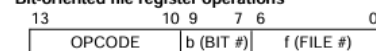
GENERAL FORMAT FOR INSTRUCTIONS

Byte-oriented file register operations



d = 0 for destination W
d = 1 for destination f
f = 7-bit file register address

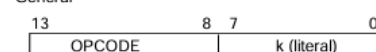
Bit-oriented file register operations



b = 3-bit bit address
f = 7-bit file register address

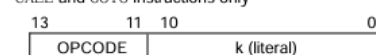
Literal and control operations

General



k = 8-bit immediate value

CALL and GOTO instructions only



k = 11-bit immediate value

TABLE 15-2: PIC16F882/883/884/886/887 INSTRUCTION SET

Mnemonic, Operands	Description	Cycles	14-Bit Opcode		Status Affected	Notes
			MSb	LSb		
BYTE-ORIENTED FILE REGISTER OPERATIONS						
ADDWF	f, d	Add W and f	1	00 0111 dfff ffff	C, DC, Z	1, 2
ANDWF	f, d	AND W with f	1	00 0101 dfff ffff	Z	1, 2
CLRF	f	Clear f	1	00 0001 1fff ffff	Z	2
CLRWF	—	Clear W	1	00 0001 0xxxx xxxxx	Z	
COMF	f, d	Complement f	1	00 1001 dfff ffff	Z	1, 2
DECf	f, d	Decrement f	1	00 0011 dfff ffff	Z	1, 2
DECFSZ	f, d	Decrement f, Skip if 0	1(2)	00 1011 dfff ffff		1, 2, 3
INCF	f, d	Increment f	1	00 1010 dfff ffff	Z	1, 2
INCFSZ	f, d	Increment f, Skip if 0	1(2)	00 1111 dfff ffff		1, 2, 3
IORWF	f, d	Inclusive OR W with f	1	00 0100 dfff ffff	Z	1, 2
MOVF	f, d	Move f	1	00 1000 dfff ffff	Z	1, 2
MOVWF	f	Move W to f	1	00 0000 1fff ffff		
NOP	—	No Operation	1	00 0000 0xxx 0000		
RLF	f, d	Rotate Left f through Carry	1	00 1101 dfff ffff	C	1, 2
RRF	f, d	Rotate Right f through Carry	1	00 1100 dfff ffff	C	1, 2
SUBWF	f, d	Subtract W from f	1	00 0010 dfff ffff	C, DC, Z	1, 2
SWAPF	f, d	Swap nibbles in f	1	00 1110 dfff ffff		1, 2
XORWF	f, d	Exclusive OR W with f	1	00 0110 dfff ffff	Z	1, 2
BIT-ORIENTED FILE REGISTER OPERATIONS						
BCF	f, b	Bit Clear f	1	01 00bb bfff ffff		1, 2
BSF	f, b	Bit Set f	1	01 01bb bfff ffff		1, 2
BTFSZ	f, b	Bit Test f, Skip if Clear	1 (2)	01 10bb bfff ffff		3
BTFSZ	f, b	Bit Test f, Skip if Set	1 (2)	01 11bb bfff ffff		3
LITERAL AND CONTROL OPERATIONS						
ADDLW	k	Add literal and W	1	11 111x kkkk kkkk	C, DC, Z	
ANDLW	k	AND literal with W	1	11 1001 kkkk kkkk	Z	
CALL	k	Call Subroutine	2	10 0xxx kkkk kkkk		
CLRWDI	—	Clear Watchdog Timer	1	00 0000 0110 0100	$\overline{TO}$, $\overline{PD}$	
GOTO	k	Go to address	2	10 1xxx kkkk kkkk		
IORLW	k	Inclusive OR literal with W	1	11 1000 kkkk kkkk	Z	
MOVLW	k	Move literal to W	1	11 0xxx kkkk kkkk		
RETFIE	—	Return from interrupt	2	00 0000 0000 1001		
RETLW	k	Return with literal in W	2	11 01xx kkkk kkkk		
RETURN	—	Return from Subroutine	2	00 0000 0000 1000		
SLEEP	—	Go into Standby mode	1	00 0000 0110 0011	$\overline{TO}$, $\overline{PD}$	
SUBLW	k	Subtract k from literal	1	11 110x kkkk kkkk	C, DC, Z	
XORLW	k	Exclusive OR literal with W	1	11 1010 kkkk kkkk	Z	

INSTRUCCIONES PARA PROGRAMAR EL PIC16F887

- ✓ Arquitectura RISC (Reduced Instruction Set Computing)
- ✓ 35 instrucciones en total
- ✓ Las instrucciones se ejecutan en un sólo ciclo de instrucción (T_c) excepto las de salto
- ✓ Cada T_c se completa con 4 ciclos del Oscilador (T_o)
- ✓ Como la mayor frecuencia que acepta el μP es 20 MHz el menor tiempo que tarda en ejecutarse una instrucción es 200 ns

- 1: When an I/O register is modified as a function of itself (e.g., $MOVWF$ $OP10$, 1), the value used will be that value present on the pins themselves. For example, if the data latch is '1' for a pin configured as input and is driven low by an external device, the data will be written back with a '0'.
- 2: If this instruction is executed on the TMR0 register (and where applicable, d = 1), the prescaler will be cleared if assigned to the Timer0 module.
- 3: If the Program Counter (PC) is modified, or a conditional test is true, the instruction requires two cycles. The second cycle is executed as a NOP.

ORGANIZACIÓN DE LA MEMORIA RAM en el PIC16F887

- ✓ Capacidad: 368 posiciones x 8 bits
- ✓ Organizada en 4 bancos (0, 1, 2 y 3)
- ✓ Posee 2 tipos de direccionamiento: directo e indirecto
- ✓ Cada posición de 8 bits se llama Registro
- ✓ A cada posición se accede con 9 bits
- ✓ Hay 2 tipos de Registros: de Funciones Especiales (SFR) y de Usos Generales
- ✓ Los Registros importantes se acceden desde los 4 bancos
- ✓ Tiene una zona común en Registros Generales que se accede desde los 4 bancos

File Address	File Address	File Address	File Address
Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾
00h TMR0	80h OPTION_REG	100h TMR0	180h OPTION_REG
01h PCL	81h PCL	101h PCL	181h PCL
02h STATUS	82h STATUS	102h STATUS	182h STATUS
03h FSR	83h FSR	103h FSR	183h FSR
04h PORTA	84h TRISA	104h WDTCON	184h SRCON
05h PORTB	85h TRISB	105h PORTB	185h TRISB
06h PORTC	86h TRISC	106h CM1CON0	186h BAUDCTL
07h PORTD ⁽²⁾	87h TRISE	107h CM2CON0	187h ANSEL
08h PCLATH	88h PCLATH	108h CM2CON1	188h ANSELH
09h INTCON	89h INTCON	109h PCLATH	189h PCLATH
0Ah PIR1	8Ah EEDAT	10Ah INTCON	18Ah INTCON
0Bh PIR2	8Bh EEDAT	10Bh EECON1	18Bh EECON1
0Ch TMR1L	8Ch EEDATH	10Ch EECON2 ⁽¹⁾	18Ch Reserved
0Dh TMR1H	8Dh EEDATH	10Dh Reserved	18Dh Reserved
0Eh TMR2	8Eh OSCCON	10Eh Reserved	18Eh Reserved
0Fh TMR2	8Fh OSCCON	10Fh Reserved	18Fh Reserved
10h TZCON	90h SSPCON2	110h General Purpose Registers	190h General Purpose Registers
11h SSPBUF	91h PR2	111h General Purpose Registers	191h General Purpose Registers
12h SSPCON	92h SSPADD	112h General Purpose Registers	192h General Purpose Registers
13h SSPCON	93h SSPSTAT	113h General Purpose Registers	193h General Purpose Registers
14h CCP1IL	94h WPUB	114h General Purpose Registers	194h General Purpose Registers
15h CCP1IH	95h IOCB	115h General Purpose Registers	195h General Purpose Registers
16h CCP1CON	96h VRCON	116h General Purpose Registers	196h General Purpose Registers
17h RCSTA	97h TXSTA	117h General Purpose Registers	197h General Purpose Registers
18h TXREG	98h SPBRG	118h General Purpose Registers	198h General Purpose Registers
19h RCREG	99h SPBRGH	119h General Purpose Registers	199h General Purpose Registers
1Ah CCP2L	9Ah PWM1CON	120h General Purpose Registers	1A0h General Purpose Registers
1Bh CCP2H	9Bh ECCPAS	121h General Purpose Registers	1A1h General Purpose Registers
1Ch CCP2CON	9Ch PSTRCON	122h General Purpose Registers	1A2h General Purpose Registers
1Dh ADRESH	9Dh ADRESL	123h General Purpose Registers	1A3h General Purpose Registers
1Eh ADCON0	9Eh ADCON1	124h General Purpose Registers	1A4h General Purpose Registers
1Fh		125h General Purpose Registers	1A5h General Purpose Registers
20h		126h General Purpose Registers	1A6h General Purpose Registers
21h		127h General Purpose Registers	1A7h General Purpose Registers
22h		128h General Purpose Registers	1A8h General Purpose Registers
23h		129h General Purpose Registers	1A9h General Purpose Registers
24h		130h General Purpose Registers	1AAh General Purpose Registers
25h		131h General Purpose Registers	1ABh General Purpose Registers
26h		132h General Purpose Registers	1ACh General Purpose Registers
27h		133h General Purpose Registers	1ADh General Purpose Registers
28h		134h General Purpose Registers	1AEh General Purpose Registers
29h		135h General Purpose Registers	1AFh General Purpose Registers
2Ah		136h General Purpose Registers	1B0h General Purpose Registers
2Bh		137h General Purpose Registers	1B1h General Purpose Registers
2Ch		138h General Purpose Registers	1B2h General Purpose Registers
2Dh		139h General Purpose Registers	1B3h General Purpose Registers
2Eh		140h General Purpose Registers	1B4h General Purpose Registers
2Fh		141h General Purpose Registers	1B5h General Purpose Registers
30h		142h General Purpose Registers	1B6h General Purpose Registers
31h		143h General Purpose Registers	1B7h General Purpose Registers
32h		144h General Purpose Registers	1B8h General Purpose Registers
33h		145h General Purpose Registers	1B9h General Purpose Registers
34h		146h General Purpose Registers	1BAh General Purpose Registers
35h		147h General Purpose Registers	1BBh General Purpose Registers
36h		148h General Purpose Registers	1BCh General Purpose Registers
37h		149h General Purpose Registers	1BDh General Purpose Registers
38h		150h General Purpose Registers	1BEh General Purpose Registers
39h		151h General Purpose Registers	1BFh General Purpose Registers
3Ah		152h General Purpose Registers	1C0h General Purpose Registers
3Bh		153h General Purpose Registers	1C1h General Purpose Registers
3Ch		154h General Purpose Registers	1C2h General Purpose Registers
3Dh		155h General Purpose Registers	1C3h General Purpose Registers
3Eh		156h General Purpose Registers	1C4h General Purpose Registers
3Fh		157h General Purpose Registers	1C5h General Purpose Registers
40h		158h General Purpose Registers	1C6h General Purpose Registers
41h		159h General Purpose Registers	1C7h General Purpose Registers
42h		160h General Purpose Registers	1C8h General Purpose Registers
43h		161h General Purpose Registers	1C9h General Purpose Registers
44h		162h General Purpose Registers	1CAh General Purpose Registers
45h		163h General Purpose Registers	1CBh General Purpose Registers
46h		164h General Purpose Registers	1CCh General Purpose Registers
47h		165h General Purpose Registers	1CDh General Purpose Registers
48h		166h General Purpose Registers	1CEh General Purpose Registers
49h		167h General Purpose Registers	1CFh General Purpose Registers
4Ah		168h General Purpose Registers	1D0h General Purpose Registers
4Bh		169h General Purpose Registers	1D1h General Purpose Registers
4Ch		170h General Purpose Registers	1D2h General Purpose Registers
4Dh		171h General Purpose Registers	1D3h General Purpose Registers
4Eh		172h General Purpose Registers	1D4h General Purpose Registers
4Fh		173h General Purpose Registers	1D5h General Purpose Registers
50h		174h General Purpose Registers	1D6h General Purpose Registers
51h		175h General Purpose Registers	1D7h General Purpose Registers
52h		176h General Purpose Registers	1D8h General Purpose Registers
53h		177h General Purpose Registers	1D9h General Purpose Registers
54h		178h General Purpose Registers	1DAh General Purpose Registers
55h		179h General Purpose Registers	1DBh General Purpose Registers
56h		180h General Purpose Registers	1DCh General Purpose Registers
57h		181h General Purpose Registers	1DDh General Purpose Registers
58h		182h General Purpose Registers	1DEh General Purpose Registers
59h		183h General Purpose Registers	1DFh General Purpose Registers
5Ah		184h General Purpose Registers	1E0h General Purpose Registers
5Bh		185h General Purpose Registers	1E1h General Purpose Registers
5Ch		186h General Purpose Registers	1E2h General Purpose Registers
5Dh		187h General Purpose Registers	1E3h General Purpose Registers
5Eh		188h General Purpose Registers	1E4h General Purpose Registers
5Fh		189h General Purpose Registers	1E5h General Purpose Registers
60h		190h General Purpose Registers	1E6h General Purpose Registers
61h		191h General Purpose Registers	1E7h General Purpose Registers
62h		192h General Purpose Registers	1E8h General Purpose Registers
63h		193h General Purpose Registers	1E9h General Purpose Registers
64h		194h General Purpose Registers	1EAh General Purpose Registers
65h		195h General Purpose Registers	1EBh General Purpose Registers
66h		196h General Purpose Registers	1EC General Purpose Registers
67h		197h General Purpose Registers	1EDh General Purpose Registers
68h		198h General Purpose Registers	1EEh General Purpose Registers
69h		199h General Purpose Registers	1EFh General Purpose Registers
6Ah		200h General Purpose Registers	1F0h General Purpose Registers
6Bh		201h General Purpose Registers	1F1h General Purpose Registers
6Ch		202h General Purpose Registers	1F2h General Purpose Registers
6Dh		203h General Purpose Registers	1F3h General Purpose Registers
6Eh		204h General Purpose Registers	1F4h General Purpose Registers
6Fh		205h General Purpose Registers	1F5h General Purpose Registers
70h		206h General Purpose Registers	1F6h General Purpose Registers
71h		207h General Purpose Registers	1F7h General Purpose Registers
72h		208h General Purpose Registers	1F8h General Purpose Registers
73h		209h General Purpose Registers	1F9h General Purpose Registers
74h		210h General Purpose Registers	1FAh General Purpose Registers
75h		211h General Purpose Registers	1FBh General Purpose Registers
76h		212h General Purpose Registers	1FCh General Purpose Registers
77h		213h General Purpose Registers	1FDh General Purpose Registers
78h		214h General Purpose Registers	1FEh General Purpose Registers
79h		215h General Purpose Registers	1FFh General Purpose Registers
7Ah		216h General Purpose Registers	
7Bh		217h General Purpose Registers	
7Ch		218h General Purpose Registers	
7Dh		219h General Purpose Registers	
7Eh		220h General Purpose Registers	
7Fh		221h General Purpose Registers	

Hay dos tipos de direccionamiento:

- **Directo:** le digo específicamente a que registro quiero acceder
- **Inmediato:** Por ejemplo, digo MOVWF (20H) y le indico a dónde lo quiero cargar
- **Indirecto:** tiene 2 registros asociados, uno es el FSR y el INDF (ficticio). Esto resulta útil cuando quiero direccionar a una zona de memoria que empieza en una dirección y termina en otra. De esta manera evitamos repetir largos códigos

Ejemplo: tengo un dato en W (W 0x30) y a este lo quiero cargar en más de una dirección, desde 20H hasta 2FH. Esto sería complicado si queremos hacerlo uno por uno, debería repetir la instrucción 16 veces. Para salvar esto necesitamos definir un puntero, que en este micro está definido a partir del registro FSR y cargar el valor que queremos mover en el INDF.

Diferencia entre directo e inmediato

En el **direccionamiento inmediato**, el operando es un valor literal presente en la instrucción, mientras que en el **direccionamiento directo**, la instrucción proporciona una dirección de memoria desde donde se debe leer el valor del operando.

Para acceder a un banco por direccionamiento directo deben colocarse previamente los valores correspondientes en RP0 y RP1 y posteriormente la dirección de 7 bits del Registro.

REGISTRO STATUS:	IRP	RP1	RP0	TO	PD	Z	DC	C
------------------	-----	-----	-----	----	----	---	----	---

PORTA	05h	TRISA	85h	WDTCON	105h	SRCON	185h
PORTB	06h	TRISB	86h	PORTB	106h	TRISB	186h
PORTC	07h	TRISC	87h	CM1CON0	107h	BAUDCTL	187h
PORTD ⁽²⁾	08h	TRISD ⁽²⁾	88h	CM2CON0	108h	ANSEL	188h

¿Cómo se obtienen los 9 bits necesarios para para acceder al Registro en direccionamiento directo?

Ejemplo:

Registro PORTA: [RP1 RP0 fff fff] = **00** 000 0101 = 05H
 Registro TRISA: [RP1 RP0 fff fff] = **01** 000 0101 = 85H
 Registro WDTCON: [RP1 RP0 fff fff] = **10** 000 0101 = 105H
 Registro SRCON: [RP1 RP0 fff fff] = **11** 000 0101 = 185H

Permite seleccionar entre los 4 registros

es el mismo valor en los 4 casos

Address	Address	Address	Address
Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾
00h	00h	100h	100h
OPTION_REG	OPTION_REG	OPTION_REG	OPTION_REG
01h	01h	101h	101h
PCL	PCL	PCL	PCL
02h	02h	102h	102h
STATUS	STATUS	STATUS	STATUS
03h	03h	103h	103h
FSR	FSR	FSR	FSR
04h	04h	104h	104h
PORTA	TRISA	WDTCON	SRCON
05h	85h	105h	185h
PORTB	TRISB	PORTB	TRISB
06h	86h	106h	186h
PORTC	TRISC	CM1CON0	BAUDCTL
07h	87h	107h	187h
PORTD ⁽²⁾	TRISD ⁽²⁾	CM2CON0	ANSEL
08h	88h	108h	188h
PORTE	TRISE	CM2CON1	ANSELH
09h	89h	109h	189h
PCLATH	PCLATH	PCLATH	PCLATH
0Ah	8Ah	10Ah	18Ah
INTCON	INTCON	INTCON	INTCON
0Bh	8Bh	10Bh	18Bh
PIR1	PIE1	EEDAT	ECON1
0Ch	8Ch	10Ch	18Ch
PIR2	PIE2	EEDAT	ECON2 ⁽¹⁾
0Dh	8Dh	10Dh	18Dh
TMR1L	PCON	EEDATH	Reserved
0Eh	8Eh	10Eh	18Eh
TMR1H	OSCCON	EEDATH	Reserved
0Fh	8Fh	10Fh	18Fh
T1CON	OSCTUNE		
10h	90h	110h	190h
TMR2	SSPCON2		
11h	91h	111h	191h
T2CON	PR2		
12h	92h	112h	192h
SSPBUF	SSPAD0		
13h	93h	113h	193h
SSPCON	SSPSTAT		
14h	94h	114h	194h
CCPR1L	WPUB		
15h	95h	115h	195h
CCPR1H	IOCB		
16h	96h	116h	196h
CCP1CON	VRCON		
17h	97h	117h	197h
RCSTA	TXSTA		
18h	98h	118h	198h
TXREG	SPBRG		
19h	99h	119h	199h
RCREG	SPBRGH		
1Ah	9Ah	11Ah	19Ah
CCPR2L	PWM1CON		
1Bh	9Bh	11Bh	19Bh
CCPR2H	ECCPAS		
1Ch	9Ch	11Ch	19Ch
CCP2CON	PSTRCON		
1Dh	9Dh	11Dh	19Dh
ADRESH	ADRESL		
1Eh	9Eh	11Eh	19Eh
ADCON0	ADCON1		
1Fh	9Fh	11Fh	19Fh
	A0h	120h	1A0h
General Purpose Registers	General Purpose Registers	General Purpose Registers	General Purpose Registers
3Fh	3Fh	1EFh	1EFh
40h	40h	1F0h	1F0h
96 Bytes	96 Bytes	1F1h	1F1h
6Fh	6Fh		
70h	70h		
7Fh	7Fh		

Registros accesibles desde todos los bancos:

File Address	File Address	File Address	File Address
Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾
00h	00h	100h	100h
PCL	PCL	PCL	PCL
02h	02h	102h	102h
STATUS	STATUS	STATUS	STATUS
03h	03h	103h	103h
FSR	FSR	FSR	FSR
04h	04h	104h	104h
PCLATH	PCLATH	PCLATH	PCLATH
0Ah	0Ah	10Ah	10Ah
INTCON	INTCON	INTCON	INTCON
0Bh	0Bh	10Bh	10Bh
70h	70h	170h	170h
7Fh	7Fh	17Fh	17Fh

Ejemplo:

Registro STATUS: [RP1 RP0 fff fff] = **00** 000 0011 = 03H
 Registro STATUS: [RP1 RP0 fff fff] = **01** 000 0011 = 83H
 Registro STATUS: [RP1 RP0 fff fff] = **10** 000 0011 = 103H
 Registro STATUS: [RP1 RP0 fff fff] = **11** 000 0011 = 183H

NO HAY 4 Registro STATUS, hay sólo UNO
y se puede acceder al mismo con 4 direcciones diferentes

Contenido de los Registros de Funciones Especiales – FSR – del Banco 0

- ✓ Cada uno de 8 bits
- ✓ Pueden manejarse como bits o bytes
- ✓ Se pueden leer y escribir o sólo leer

Programando estos registros el micro se ajustará a la rutina que pretendemos que realice.

El contenido de cada uno y su programación se verá a medida que se vaya viendo cada módulo

Addr	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR, BOR
Bank 0										
00h	INDF	Addressing this location uses contents of FSR to address data memory (not a physical register)								XXXXXX XXXXXX
01h	TMR0	Timer0 Module Register								XXXXXX XXXXXX
02h	PCL	Program Counter's (PC) Least Significant Byte								0000 0000
03h	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C	0001 1XXXXX
04h	FSR	Indirect Data Memory Address Pointer								XXXXXX XXXXXX
05h	PORTA ⁽³⁾	RA7	RA6	RA5	RA4	RA3	RA2	RA1	RA0	XXXXXX XXXXXX
06h	PORTB ⁽³⁾	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0	XXXXXX XXXXXX
07h	PORTC ⁽³⁾	RC7	RC6	RC5	RC4	RC3	RC2	RC1	RC0	XXXXXX XXXXXX
08h	PORTD ^(3,4)	RD7	RD6	RD5	RD4	RD3	RD2	RD1	RD0	XXXXXX XXXXXX
09h	PORTE ⁽³⁾	—	—	—	—	RE3	RE2 ⁽⁴⁾	RE1 ⁽⁴⁾	RE0 ⁽⁴⁾	---- XXXXXX
0Ah	PCLATH	Write Buffer for upper 5 bits of Program Counter								---0 0000
0Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF ⁽¹⁾	0000 000X
0Ch	PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	-000 0000
0Dh	PIR2	OSFIF	C2IF	C1IF	EEIF	BCLIF	ULPWUIF	—	CCP2IF	0000 00-0
0Eh	TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register								XXXXXX XXXXXX
0Fh	TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register								XXXXXX XXXXXX
10h	T1CON	T1GINV	TMR1GE	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON	0000 0000
11h	TMR2	Timer2 Module Register								0000 0000
12h	T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	-000 0000
13h	SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register								XXXXXX XXXXXX
14h	SSPCON ⁽²⁾	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	0000 0000
15h	CCPR1L	Capture/Compare/PWM Register 1 Low Byte (LSB)								XXXXXX XXXXXX
16h	CCPR1H	Capture/Compare/PWM Register 1 High Byte (MSB)								XXXXXX XXXXXX
17h	CCP1CON	P1M1	P1M0	DC1B1	DC1B0	CCP1M3	CCP1M2	CCP1M1	CCP1M0	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000X
19h	TXREG	EUSART Transmit Data Register								0000 0000
1Ah	RCREG	EUSART Receive Data Register								0000 0000
1Bh	CCPR2L	Capture/Compare/PWM Register 2 Low Byte (LSB)								XXXXXX XXXXXX
1Ch	CCPR2H	Capture/Compare/PWM Register 2 High Byte (MSB)								XXXXXX XXXXXX
1Dh	CCP2CON	—	—	DC2B1	DC2B0	CCP2M3	CCP2M2	CCP2M1	CCP2M0	--00 0000
1Eh	ADRESH	A/D Result Register High Byte								XXXXXX XXXXXX
1Fh	ADCON0	ADCS1	ADCS0	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON	0000 0000

Programa Fuente:

El programa fuente está compuesto por una sucesión de líneas de programa. Tiene 4 campos bien definidos separados por espacios o tabulaciones:

[etiqueta] Comandos [Operando/s] [: comentarios]

Estructura Programa Fuente:

Las distintas **áreas que componen un programa fuente** son las que se indican en el esquema.

Tener en cuenta que está compuesto por una **sucesión de líneas de programa** donde algunas son **directivas** para el ensamblador al compila y otras son **instrucciones** del programa real que se graba en la memoria de programa.

El **programa en binario** que se graba en la memoria está en **lenguaje de máquina**.

En todos los micros el Lenguaje assembler y el compilador es **propietario**.

```

;*****
;* Programa que hace realiza .... (bla, bla, bla)
;* .....
;* El hardware consite en ....
;*
;* Autores:.... Fecha:..... Versión: .....
;*****
LIST                P=16F887
INCLUDE             <p16f887.inc>
__CONFIG __CONFIG1, _FOSC_INTRC_NOCLOCKOUT & _WDTE_OFF

VARIABLE1 EQU 0x20
VARIABLE2 EQU 0x21
CONTA EQU .14

#DEFINE BANK0 bcf,RP0 ;cambio al banco 0

MULx4 MACRO
BCF STATUS,C
RLF VARIABLE2,1
BCF STATUS,C
RLF VARIABLE2,1
ENDM

ORG 0x00
GOTO INICIO

ORG 0x04
GOTO INTERRUPTION

ORG 0x05
INICIO
BSF STATUS,RP0
BSF STATUS,RP1
MOVW CONTA, VARIABLE1 ; el contador en 14
CLR ANSEL ; ingresa las rpm a medir
MOVW TRISB, STATUS,RP1 ; puerto conectado al rotor
BCF STATUS,RP1
....
GOTO $

INTERRUPTON
....
RETFIE

END

```

Encabezado

Declaraciones generales sobre la Plataforma

Equivalencia de variables, definiciones y macros

Vectores de Inicio e Interrupción

Programa principal

Subrutinas síncronas y asíncronas

Final

Directivas del Assembler del PIC16F887

Directiva: Comando específico que le doy al compilador y no es una instrucción

Directiva LIST

Este comando sirve para que el compilador tenga en cuenta sobre qué procesador se está trabajando. Este comando debe estar en todo proyecto.

Directiva #INCLUDE

Esta directiva indica que archivos deberán tomarse en cuenta a la hora de compilar el código. Normalmente se usa para incluir el archivo de PIC que el ensamblador tiene entre sus archivos, con el cual el compilador será capaz de reconocer todos los registros especiales y sus bits. Los nombres de los SFR y sus bits son palabras reservadas y no pueden ser usadas al programar.

Directiva #DEFINE

#DEFINE se usa para crear pequeños macros. Con estas macros podremos poner nombres a pequeños fragmentos de código que nos facilitarán la realización y comprensión del algoritmo.

Ejemplo:

```
#define LED_AZUL PORTA,3 ; ahora el "nombre" del bit 3 del PORTA será LED_AZUL
y en el programa, si queremos poner ese bit a 0, no necesitamos "recordar" que bit era ya que en lugar de:
        BCF     PORTA,3
Se pondrá:
        BCF     LED_AZUL
```

Directiva EQU

La directiva EQU permite al programador "igualar" nombres personalizados a datos o direcciones. Los nombres se refieren generalmente a direcciones de dispositivos, datos numéricos, direcciones de comienzo, direcciones fijas, posiciones de bits, etc.

Ejemplo:

```
clockrate    EQU    .20      ;frecuencia del cristal
entradas     EQU    b'11010001'
fclk         EQU    clockrate/4 ;frecuencia del reloj interno
guarda_valor EQU    0x3F
```

Directiva ORG

Esta directiva dice al ensamblador a partir de qué posición de memoria de programa se situarán las siguientes instrucciones. Reset, Subrutinas, Subrutinas de interrupción y otros Tablas deben colocarse en la dirección de memoria indicada. Recordemos que el 16F887 tiene 8192 posiciones de memoria flash para código.

Ejemplo:

```
ORG    0x00      ; El programa comienza en la dirección 0 y
        GOTO    INICIO ; salta a la dirección 5 para sobrepasar
ORG    0x05      ; el vector de interrupción, situado en la posición 4
INICIO
...
ORG    0x800
TABLA
...
```

Directiva CONFIG

Este comando se utiliza en todos los PICs para aplicar una configuración especial. Su efecto es colocar determinados bits de configuración en 1 o 0 para otorgar una prestación deseada. Setea tipos de reset, watchdog timer, selecciona oscilador, etc. Estos bits están mapeados en la dirección 2007H y 2008H y mediante el & se enlazan las etiquetas de configuración.

Ejemplo:

```
__CONFIG __CONFIG1, _FOSC_INTRC_NOCLKOUT & _WDTE_OFF & _MCLRE_ON
```

Directiva BANKSEL

Esta directiva coloca los bits de direccionamiento directo (RP0, RP1) apuntando el banco que contiene al Registro invocado. El ensamblador reemplaza BANKSEL por dos instrucciones que colocan RP0 y RP1 en los valores correspondientes al banco.

Ejemplo:

```
Inicio  BSF      STATUS, RP0
        BSF      STATUS, RP1 ; direcciona Banco 3
        MOVLW    0x3B
        MOVWF    ANSEL
```

```
Inicio  BANKSEL  ANSEL ; direcciona Banco de ANSEL
        MOVLW    0x3B
        MOVWF    ANSEL
```

Directiva \$

Permite cargar el PC (Contador de Programa) para saltos cortos y sin usar etiquetas. El símbolo \$ es asignado por el ensamblador a la dirección del Program Counter de la instrucción en curso. Se le puede indicar al ensamblador que sume o reste un número para ir a hacia adelante o hacia atrás pero sólo es recomendable usar esta indexación para saltos de pocas instrucciones.

Ejemplo:

```
loop    GOTO    loop
...
loop1   NOP
        NOP
        GOTO    loop1
```

```
GOTO    $
...
NOP
NOP
GOTO    $-2
```

Directiva END

Este comando se utiliza para indicarle al compilador que terminó el programa fuente. Es indispensable colocarlo al final del programa o el ensamblador dará error. Tener en cuenta que es una directiva de assembler NO una instrucción, no tiene ningún efecto sobre el lenguaje de máquina.

[DIGITAL II TEORICO 5 abril 2024](#)

PUERTOS

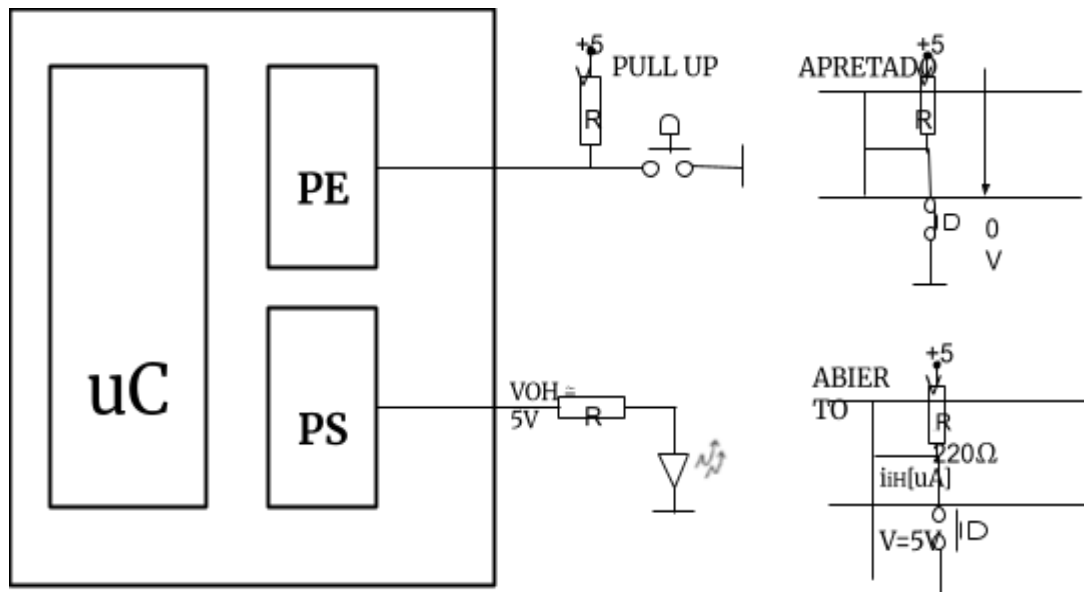
Un puerto no es más que un lugar en dónde entran y salen datos del mundo exterior, éstos son en su mayoría de 8 bits.

Tengo mi microcontrolador y necesito implementar un puerto de entrada y un puerto de salida. Por ejemplo, si necesito testear un pulsador: el pulsador entra a un pin del puerto de entrada, tengo que ser capaz de decirle al PIC si el pulsador está apretado o no. La forma más fácil es darle un valor de tensión (apretado→ 0 y no apretado→ 1).

Si el pulsador no está apretado el pin de entrada tiene un 1 dado, por una resistencia de pull up a +5v (si esta resistencia no estuviera no tendríamos un valor fijo). Por el contrario, si está apretado se lee un 0.

Es importante considerar el rebote de la tecla, el mismo se debe evitar por hardware o software.

El valor de la resistencia pull up no es relevante. Ya que la caída de tensión que me generará es irrelevante, debido a que la iih es muy pequeña.

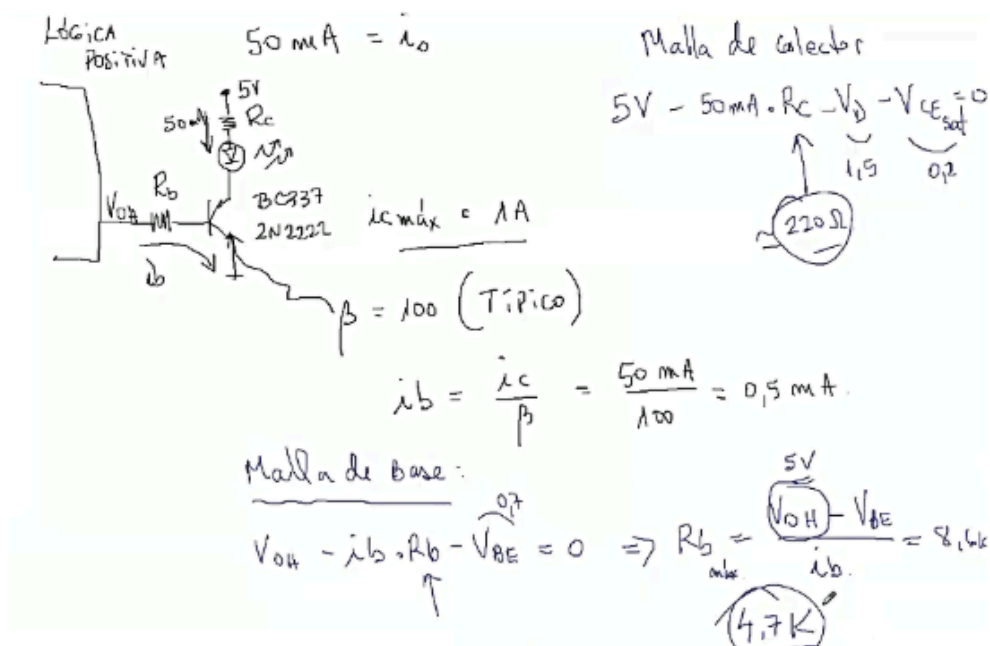


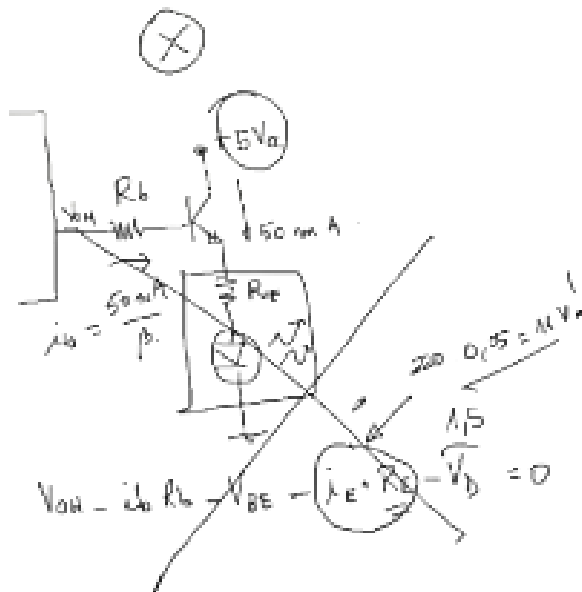
Para ésto utilizamos 1 de 8 bits, quedan 7 disponibles. Supongamos ahora, que el puerto de salida necesita prender un led, necesitamos calcular la resistencia para que no supere los valores máximos de corriente (25 mA). Planteo mallas y calculo corriente (generalmente el cálculo me da una corriente razonable de 10mA).

$$\frac{5V-1,5V}{R} = i_{OH} \quad \rightarrow R = \frac{5V-1,5V}{10mA} \quad \rightarrow R = 350\Omega \cong 330\Omega$$

Si supero la corriente máxima se debe colocar un transistor bipolar. Ahora tenemos que calcular 2 resistencias y plantear 2 ecuaciones. Lógica positiva

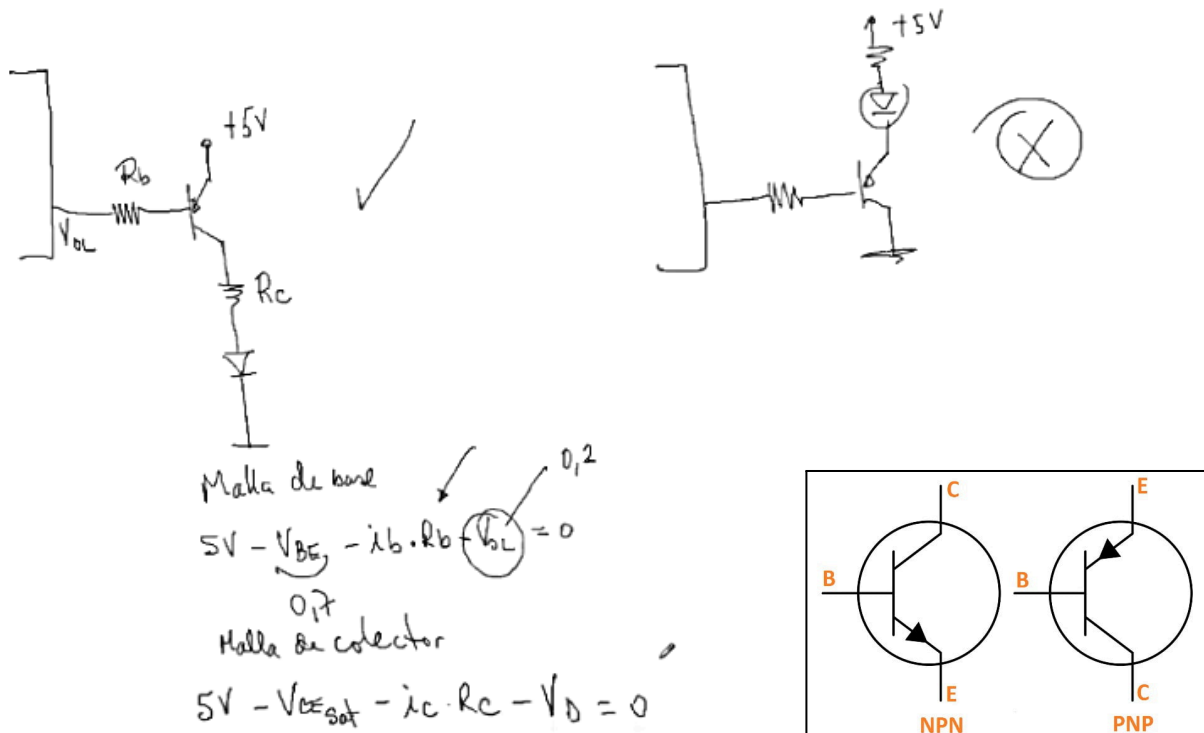
- 1 ecuación para malla de colector con factores conocidos ($V_{cesat}=0,2$ – $V_d=1,5$ Corriente y V_{cc} dato) y despejó R_C
- 2 ecuación para malla de base ($V_{BE}=0,7$ y $V_{OH}=5V$ por fabricante) y despejó R_B max





Configuración no recomendada - AL colocar las cargas en el emisor tenemos mucha caída.

Para encender por **lógica negativa**, debemos utilizar un **transistor PNP** colocado de la primera forma (**las cargas NO pueden ir en el emisor**, ni para PNP ni par NPN)



Normalmente conviene encender por bajo y no por alto, porque siempre me conviene absorber corriente y no suministrar.

Un **puerto de entrada** es generalmente un chip de tipo **buffer** con **TRI STATE**.

Buffer con tri state/ 3 estados

Es un dispositivo electrónico que puede estar en uno de tres estados diferentes: alto, bajo, o de alta impedancia. Este tercer estado, la alta impedancia, es como si el buffer estuviera desconectado del circuito, lo que permite que otras señales compartan la misma línea sin interferencia.

Estados del Buffer Tri-State

- Alto (1): El buffer envía una señal alta (1) a la salida.
- Bajo (0): El buffer envía una señal baja (0) a la salida.
- Alta Impedancia (Z): La salida del buffer está efectivamente desconectada del circuito, lo que permite que otras señales puedan utilizar la misma línea sin conflicto.

Funcionamiento Básico

El buffer tri-state tiene una entrada de control adicional llamada "enable" . Cuando esta señal de control está activa, el buffer funcionará normalmente (enviando 0 o 1). Cuando la señal de control está inactiva, el buffer entra en estado de alta impedancia.

Por ejemplo:

- Si la entrada de datos es 1 y la señal de control (enable) está activa, la salida será 1.
- Si la entrada de datos es 0 y la señal de control está activa, la salida será 0.
- Si la señal de control está inactiva, la salida estará en alta impedancia, independientemente de la entrada de datos.

A los datos que entran debemos ponerlos en alta impedancia cuando estoy sacando datos a otro lado, porque sino vamos a tener un conflicto, por un lado vamos a estar enviando un dato y por el otro va a estar ingresando otro dato. Esto es lo mismo que sucede con las memorias, que al seleccionarlás su salida se pone en estado activo, pero cuando se selecciona se pone en alta impedancia.

Un **puerto de salida** es típicamente un **LATCH**.

LATCH

Un latch (o biestable) es un tipo de circuito digital de almacenamiento que **mantiene su estado hasta que recibe una señal para cambiarlo**. A diferencia de un flip-flop, que se actualiza en función de un reloj, un latch se actualiza en función de las señales de entrada.

Cuando se dice que un puerto de salida es típicamente un latch, significa que la salida del puerto puede mantener su estado hasta que reciba una señal para cambiarlo. Esto es útil en sistemas donde se necesita que la salida mantenga su valor estable hasta que un evento específico ocurra.

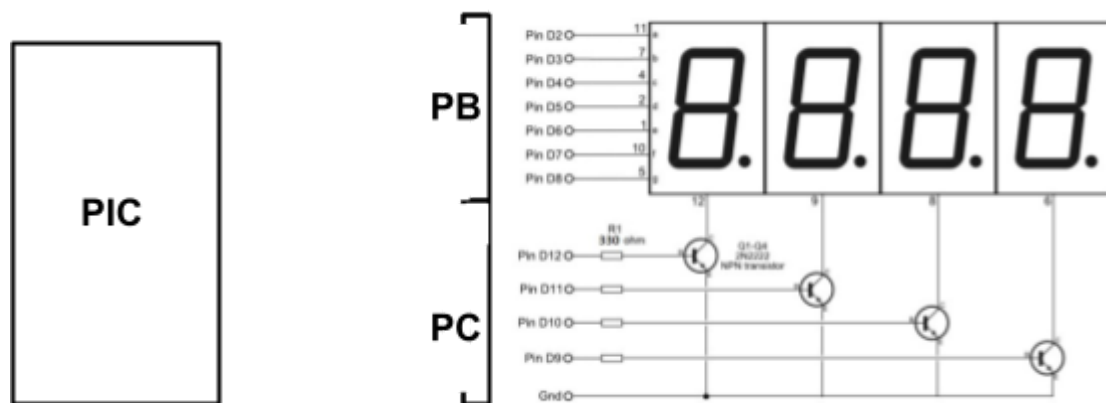
El PIC no tiene un puerto de entrada y otro de salida, cada puerto puede ser definido como entrada o como salida. Además, no hace falta que defina todo el puerto completo para una u otra cosa, sino que puedo definir algunos pines como entrada y otros como salida.

Puertos de Entrada/Salida

- Pines de entrada/salida (I/O) de propósito general
- Mediante ellos, el micro PIC puede monitorizar y controlar otros dispositivos.
- Para añadir flexibilidad al micro, muchos de sus pines de entrada/salida están multiplexados con funciones alternativas.
- Para la mayoría de los puertos, la dirección del pin I/O es controlada por el registro de dirección de datos, llamado TRIS. $TRISX<n>$ controla la dirección del pin n del puerto X.
- Un 1 en el registro TRIS indica que el pin es **entrada**. Un 0 indica que es salida.

[hcb-zrya-yvg \(2024-04-12 08:34 GMT-3\)](#)

Multiplexado Display 7 segmentos



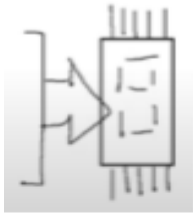
Yo dispongo de un PIC, y quiero por ej, atender un teclado y al mismo tiempo estar gestionando un display y tener otras entradas y salidas para otras funciones. No me alcanzan los puertos.

¿Con qué técnica puedo hacer un arreglo para con pocos puertos poder manejar varias cosas?

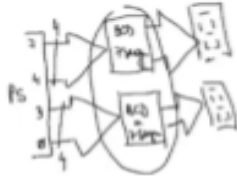
Concretamente se mandan señales que no son constantes en el tiempo, por ejemplo, este primer display va a estar encendido durante un cierto tiempo, luego se apaga y se prende el segundo display durante cierto tiempo, luego se apaga y se prende el tercero, luego se prende el cuarto y luego se vuelve a prender el primero. Es una secuencia continua, de tal forma que al ojo le parezca que está permanentemente puesto ahí sin embargo en realidad se están prendiendo y apagando una cierta cantidad de veces en función del tiempo.

Con este multiplexado, podemos manejar con pocos puertos muchas cosas.

Hay dos tipos de display, de cátodo común y de ánodo común, ambos necesitan información en sus segmentos, esta información se la doy a través de pines, los cuales deben ser controlados por el microcontrolador.



Primer opción
Con un puerto manejo solo un Display
Para representar 8 display, necesito 8 puertos

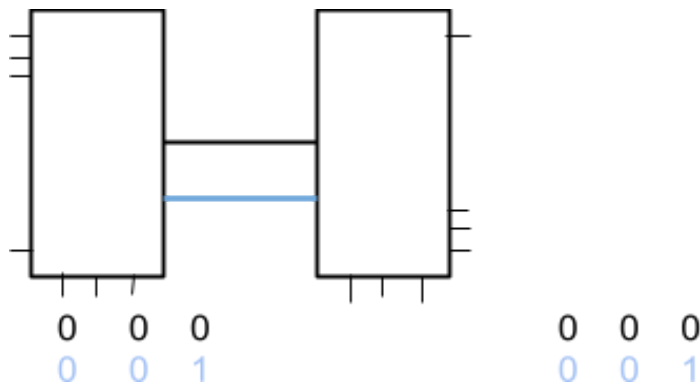


Segunda opción
Al agregar dos chips, decodificadores especiales BCD, con un puerto puedo manejar 2 display. Para representar 8 display necesito como mínimo 4 puertos.

Tercera opción

Mediante multiplexado puedo manejar hasta 8 display con solo un puerto

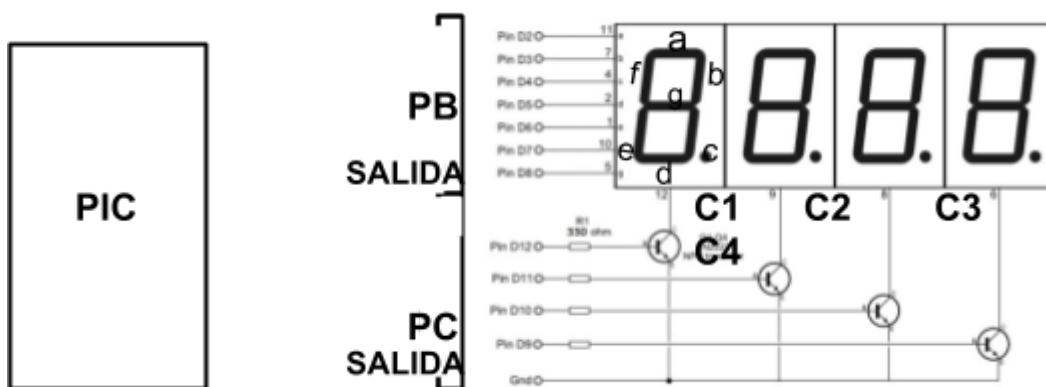
MULTIPLEXADO

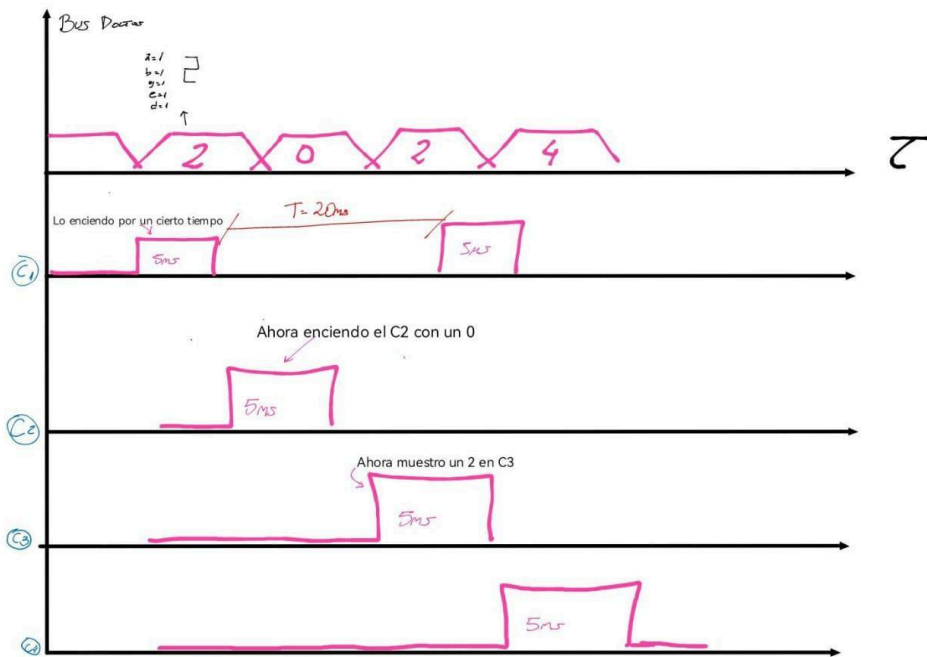


Tengo 8 señales, y la distancia entre los dos dispositivos es muy larga, no quiero tirar 8 cables porque es muy caro, quiero tirar solo 1.

Para esto debo colocar un multiplexor. El problema es que la información no esta siempre presente, lo que esta en negro esta presente por un instante de tiempo, pero cuando esta lo azul desaparece lo negro, tengo que sincronizar la información.

Si yo me preocupo para que la información se presente al menos 50 veces por segundo, la voy a ver como continua.





Por los pines coloco la información que requiere cada uno de los display para encenderse. Queremos colocar en nuestro display el número 2024.

En el eje del tiempo coloco los respectivos números que quiero mostrar. Los ejes inferiores corresponden a cada display, debo ir colocando cajones que representan que se activa cada display, según corresponda.

Por ejemplo, para encender el primer número dos, además de colocar el número sobre el ej t, debo poner el cajón de 5ms en ese momento.

Debo repetir la secuencia al menos 50 veces por segundo para que mi ojo lo vea como continuo, si no lo repito

El período T mínimo que debo tener es de 20 ms, $\frac{1}{50} = 0,02 \text{ seg} = 20 \text{ ms}$, para que esto funcione y no se vea titilando.

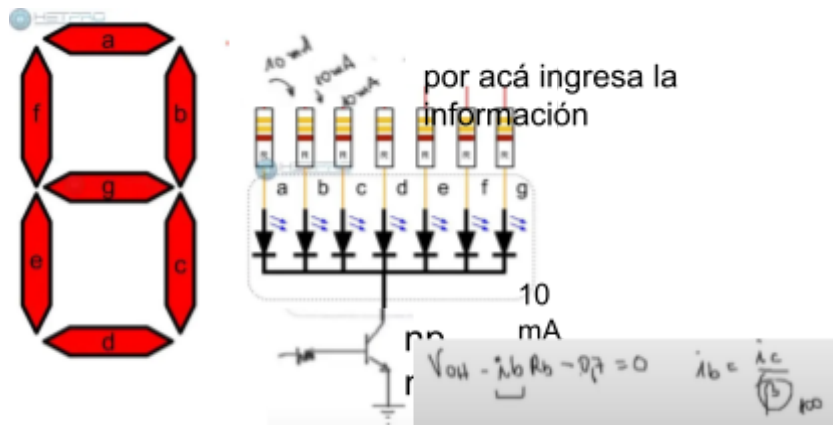
El brillo que voy a ver en cada uno de los display depende de la frecuencia y del tiempo en que esté encendido. Mientras más tiempo esté encendido el display mayor será el brillo

Si yo en lugar de tener 4 display, tuviese 8, debería disminuir el tiempo de encendido de cada uno

HAY DOS TIPOS DE DISPLAY

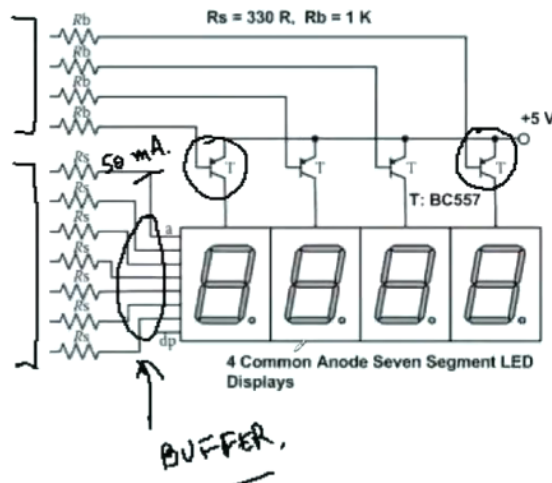
Cátodo común

- Comparten masa
- El transistor controla a todos al mismo tiempo, funciona como una llave.
- Funciona con lógica positiva al tener conectado con un **transistor npn**
- Se aconseja no encender un display de cátodo común con un transistor pnp,
- Le hago circular aprox 10 mA a cada segmento, por lo que tenemos una corriente total de 70 mA
- La idea es que se sature con la mínima corriente, de esta manera la mayor va a estar bien saturada. Hago el cálculo con la corriente mínima, de 10 mA.
- En casos de necesitar amplificar la corriente debemos conectar un **transistor pnp** a cada segmento en la parte superior.



Ánodo común

- Comparten vcc
- Debemos activarlo como un **transistor pnp**
- Puede haber casos en que la corriente del puerto no me alcance, ya que estos entregan como máximo 25 mA, para solucionar esto debemos agregar un buffer hecho con transistores npn.



Multiplexado de Teclado:

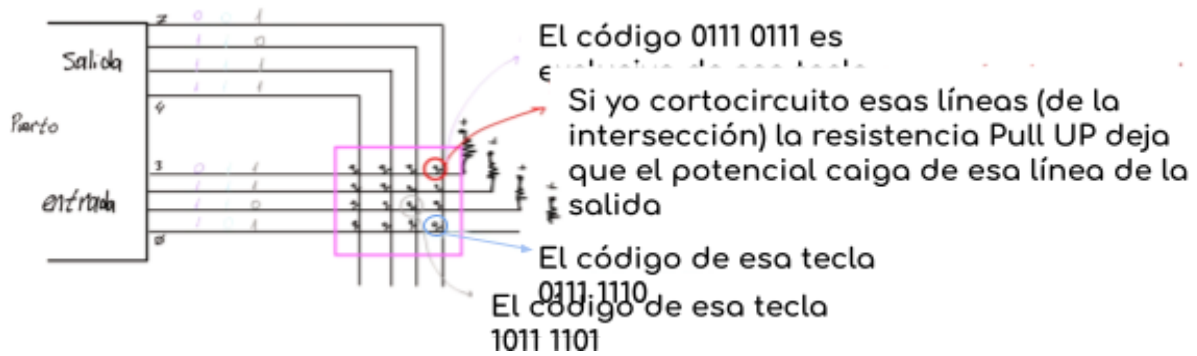
Los teclados característicos vienen de 16 teclas. *¿Cómo se hacen para leer 16 teclas?*

La forma de fuerza bruta sería colocar dos puertos de 8 y hacer entrar 8 teclas en un puerto y las 8 restantes en el otro. Al hacer esto, y no utilizar multiplexado, estaríamos utilizando 2 puertos para las 16 teclas y otros 4 puertos más para los 4 display que quisiera representar y ya me quedo sin puertos.

Una de las formas básicas que ya, no es la que se usa, pero es utilizar un **multiplexor**. Aca tenemos la posibilidad de introducir 16 teclas y con 4 pines, que van a venir de algún lugar del micro, podemos manejar esta información. Le damos la combinación que queramos, y leemos la tecla correspondiente, por lo que tenemos la ventaja de saber que tecla estamos leyendo, ya la tenemos codificada.

La gran ventaja que tenemos, es que ya tengo el dato codificado, los datos que voy a tener es directamente el número de tecla que yo aprieto, y la desventaja de esto es que debemos agregar un chip extra.

Nosotros vamos a trabajar con el teclado matricial, de esta manera **con un único puerto podemos manejar 16 teclas**, cuando antes necesitábamos como 2 puertos completos.



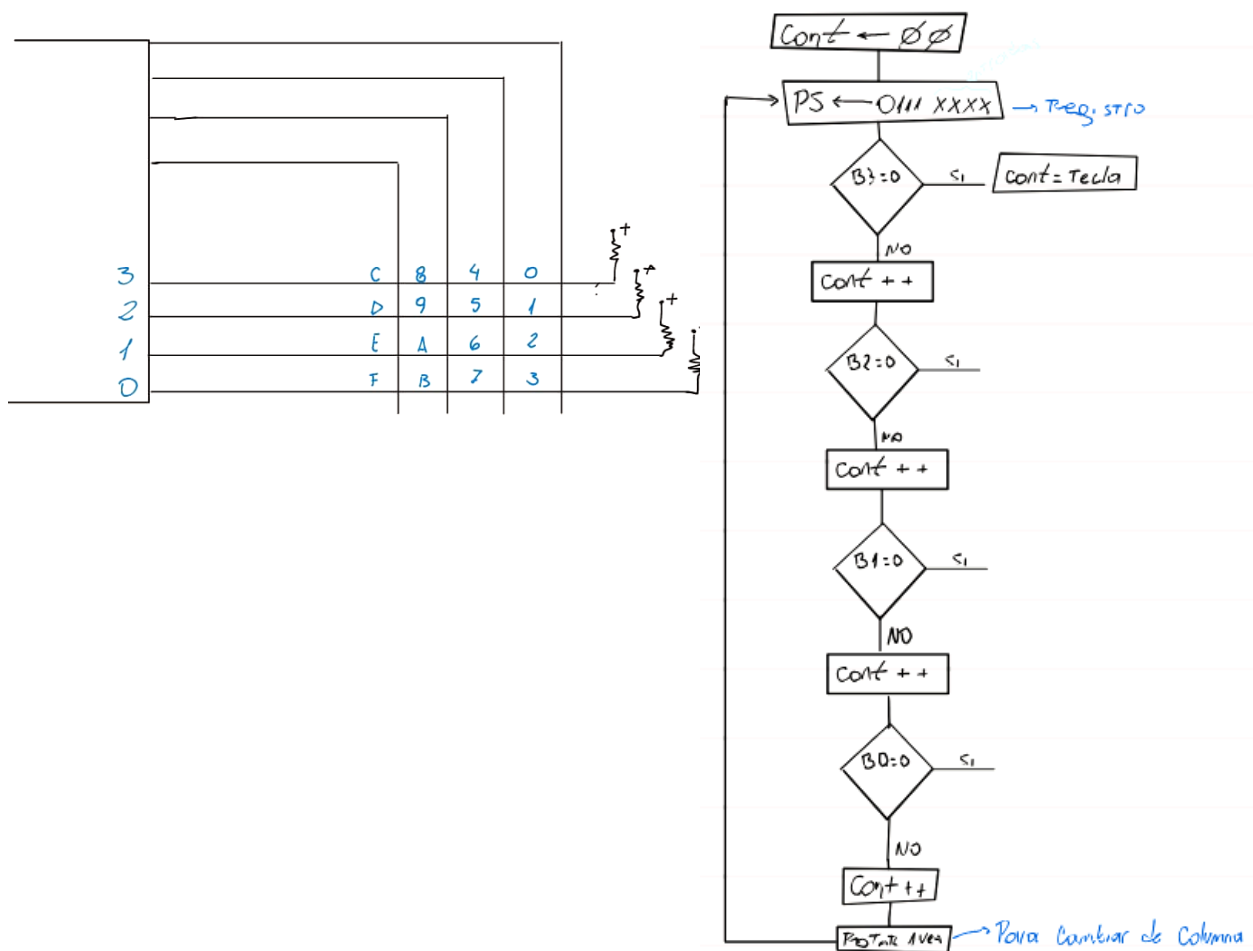
Cada tecla está asociada a un código único de bits correspondientes a la entrada y a la salida

Lo que hace cada una de las teclas es cortocircuito el hilo vertical con el horizontal.

¿Cómo lo uso para que mi sistema lea una tecla? Así como esta no me permite decodificar la tecla de manera sencilla, habría que encontrar una lógica para poder leer la tecla que yo apriete.

La desventaja de esto es que este código no sigue una secuencia que tenga una relación. Por lo tanto, para detectar de qué tecla se trata debemos hacer una tabla, la cual no sigue una secuencia lógica, tiene valores saltados en un rango de 256 valores.

¿Cómo se logra una mejora de esto? Complicando el programa, pero de esta manera evitamos el uso de la tabla, simplificandonos el hardware y la decodificación



Rebote

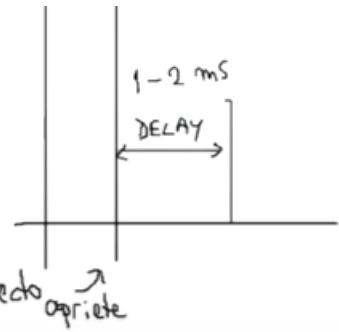
Hay dos maneras de eliminar el rebote de tecla:

- **Hardware** (no se usa)
- **Software:**

Lo que se hace básicamente es una temporización.

Una vez que detecto que la tecla fue apretada, lo que hago es esperar 1-2 ms, y lo que hace es esperar en definitiva que la tecla deje de rebotar.

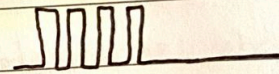
Entonces cada vez que se aprieta la tecla yo tengo que hacer un delay para asegurarme que deje de rebotar y después recién hacer lo que yo haya decidido que quiera hacer

**REBOTE**

Fenomeno mecánico que ocurre al presionar una tecla

→ los contactos metálicos dentro del interruptor pueden hacer contacto y separarse varias veces rápidamente antes de estabilizarse.

→ Puede interpretarse erróneamente como múltiples pulsaciones de tecla en lugar de una sola.



Hay 2 maneras de eliminar el rebote de tecla

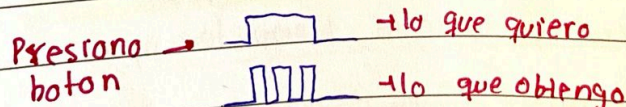
→ **Hardware:**

• Implica el uso de capacitores o filtros para suavizar las transiciones de contacto dentro del interruptor.

→ **Software:**

• Mediante algoritmos que ignoran las pulsaciones repetidas

→ Delay (1ms o 2ms) después de la detección de la pulsación inicial



INTERRUPCIONES

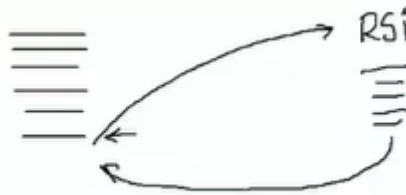
Las interrupciones son desviaciones del flujo de control del programa originadas asincrónicamente por diversos sucesos que no se hallan bajo la supervisión de las instrucciones. Dichos sucesos pueden ser externos al sistema, con la generación de un flanco o nivel activo en una patilla del microcontrolador o bien, internos, como el desbordamiento de un contador. Su comportamiento es similar al de la instrucción "call" de llamado a subrutina. Se detiene la ejecución del programa en curso, se salva la dirección actual del contador de programa (PC) en la pila (STACK) y se carga el PC con una dirección, que en el caso de una interrupción es una dirección reservada de la memoria de código, llamada vector de interrupción.

En el PIC, este vector de interrupción está ubicado en la posición 04H, en donde comenzará la rutina de servicio de la interrupción.

Ejemplo:

org OOH : Vector de origen del programa
 goto INICIO; salto a la rutina de programa principal
 org 04H : Vector de interrupción
 goto ISR; salto a la rutina de interrupción.

La idea de colocar una instrucción de salto incondicional, es la de trasladar el flujo del programa a la zona de memoria de código que contiene a la rutina de interrupción.



Yo estoy realizando una instrucción tras instrucción, y de repente viene un pedido de interrupción. El microcontrolador almacena la posición desde la cual está saliendo, va a atender la ISR (RSI, Rutina de Servicio de Interrupción), atiende lo que hay que hacer allí y luego vuelve con una instrucción específica. Hay una serie de registros y bits que hay que manejar para que esto ocurra adecuadamente.

Es como si fuera una subrutina donde, en lugar de llamarla con un call dentro del programa se la llama externamente con algún evento.

Los eventos que veremos son 3, los asociados al activación del Pin **RB0/INT**, desbordamiento del **TMRO** y cambio de estado de uno de los pines del **puerto B**.

DIAGRAMA DE FLUJO DURANTE UNA INTERRUPCIÓN

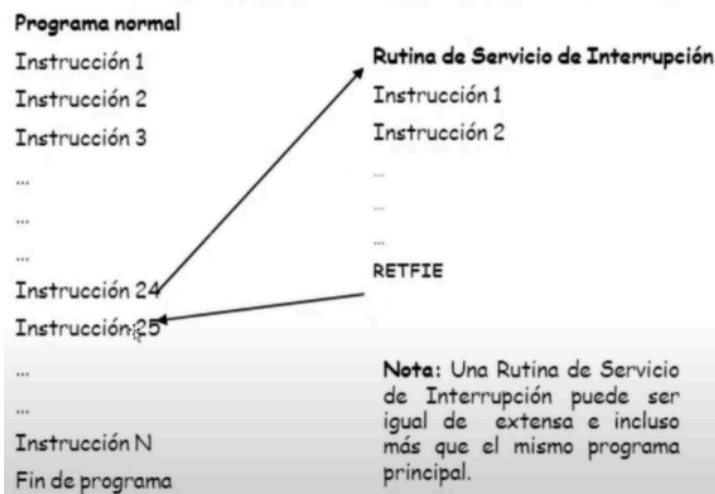
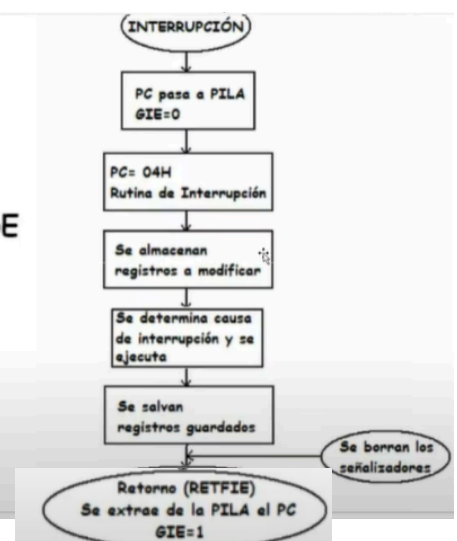
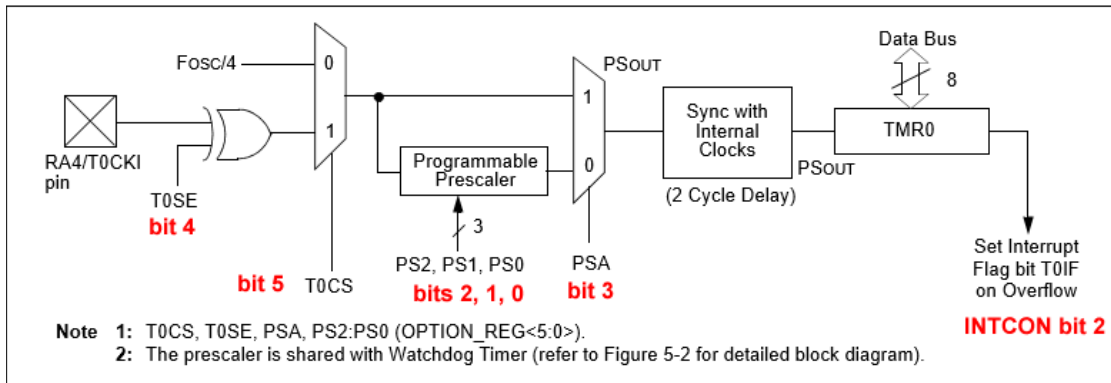


DIAGRAMA DE FLUJO RSI



REGISTRO TMR0

Es el corazón del módulo Timer0. Puede ser leído o escrito en cualquier momento. El TMR0 se incrementará automáticamente por cada ciclo de instrucción y contará desde 0(00H) hasta 255(FFH)

FIGURE 5-1: TMR0 BLOCK DIAGRAMwww.bristolwatch.com

Tenemos dos fuentes posibles de cuenta, por un pin externo o con la frecuencia del oscilador, elegimos cuál utilizar con la pata de control T0CS. A partir de ahí yo puedo, o no, asignar un prescaler con el pin de selección PSA. Luego de un bloque de hardware interno del uc pasa a contar. **¿cuando genero una interrupción?** Cuando TMR0 desborda, es decir pasa de FFH a 00H.

Registros Asociados al temporizador TMR0

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR, BOR	Value on all other RESETS
01h	TMR0	Timer0 Module Register								XXXX XXXX	XXXX XXXX
0Bh,8Bh	INTCON	GIE	EEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
51h	OPTION_REG	RBPV	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111
85h	TRISA	—	—	—	PORTA Data Direction Register					---1 1111	---1 1111

Legend: x = unknown, u = unchanged, - = unimplemented locations read as '1'. Shaded cells are not used by Timer0.

- **TMR0**: Registro de 8 bits de lectura/escritura
- **OPTION_REG**: Configura al TMR0 para que trabaje como temporizador o contador y asigna el valor al prescaler.
- **INTCON**: Da información mediante el bit "TOIF" cuando el TMR0 se ha desbordado.
- **TRISA(PUERTO A)**: Permite el ingreso de pulsos cuando el TMR0 está configurado como contador por RA4/T0CKI

¿Cómo cuenta el TMR0?

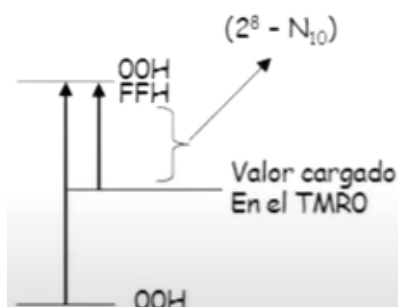
El TMR0 cuenta **exclusivamente de forma ascendente**, nunca descendente.

¿Quién incrementa al TMR0?

Cada instrucción que se ejecuta en un programa se encarga de incrementarlo

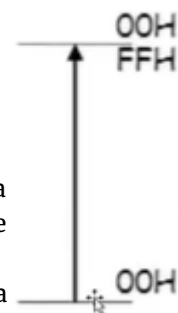
Si yo quiero contar hasta un número distinto de 256, entre 00H y FFH, el TMR0 se carga con un valor inicial, el TMR0 comenzará a contar desde el valor cargado hasta que se desborda (cuando pasa a 00H).

Debemos cargar este número cada vez que se largue la cuenta, porque si lo hacemos una sola vez las veces posteriores no va a funcionar



Ej: Quiero contar hasta 50

Para esto, debo precargar este registro, que cuenta hasta 256, con 206. De tal forma que las siguientes 50 cuentas hagan que desborde y en ese momento dé la interrupción.



¿Qué es el PRESCALER?

- Nos permite contar hasta un número diferente del estipulado por los 255.
- Divide la frecuencia de reloj de entrada del Timer0, entre valores predefinidos, como se ve en la tabla asociada al registro OPTION, 1:32, 1:64, 1:256, etc., genera una nueva señal de menor frecuencia a la salida, que será la señal de reloj de entrada al registro TMRO.
- “Ralentiza” señales de entrada demasiado rápidas para nuestros propósitos.
- También existe un postescaler, asociado al perro guardián WDT del uc, pero en este caso recibe el nombre de postcaler ya que se usa a la salida del WDT, no pudiendo estar asignado a la vez al Timer0 al escribir sobre él, pero se puede seleccionar por software, como ya se ha dicho, mediante el registro OPTION.
- **Nota:** Para evitar un RESET no deseado al sistema, es necesario ejecutar una secuencia de instrucciones específicas cuando se cambia la asignación del prescaler del TMRO al WDT. Esta secuencia debe ser seguida, aun cuando el WDT esté inactivo

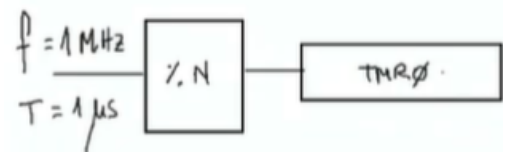
REGISTER 2-2: OPTION_REG: OPTION REGISTER

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
RBPV	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

El prescaler está manejado con esta tabla, en los últimos 3 bits del registro OPTION.

Podemos asignar el prescaler tanto al TMRO como al WDT, y solo podemos dividir por estos números.

Bit Value	Timer0 Rate	WDT Rate
000	1 : 2	1 : 1
001	1 : 4	1 : 2
010	1 : 8	1 : 4
011	1 : 16	1 : 8
100	1 : 32	1 : 16
101	1 : 64	1 : 32
110	1 : 128	1 : 64
111	1 : 256	1 : 128



Sugerencia: Hacer estos cálculos a mano con la siguiente lógica

Si quiero contar, por ejemplo, hasta $t=10\text{ ms}$ con $f=1\text{ MHz}$ debo ver cuantas cuentas debo hacer.

$$\frac{t}{T} = \frac{10 \times 10^{-3}}{1 \times 10^{-6}} = 10.000$$

Tengo que realizar 10.000 cuentas, las cuales evidentemente no van a entrar dentro de las 255, entonces tenemos que jugar un poco con la división %N

$$\frac{1\text{ MHz}}{16} = 62.500\text{ Hz}, T=16\text{ us}$$

Esta frecuencia de 62.500Hz es la que voy a tener luego de %N, ahora tenemos

$$\frac{10 \times 10^{-3}}{16 \times 10^{-6}} = 625$$

Este valor de %N no nos sirve ya que $625 > 255$, por lo que probamos dividir por 32.

$$\frac{1\text{ MHz}}{32} = 31.250\text{ Hz}, T=32\text{ us}$$

$$\frac{10 \times 10^{-3}}{32 \times 10^{-6}} = 312,5$$

Observamos que este valor tampoco nos sirve, probamos ahora dividir por 64.

$$\frac{1\text{ MHz}}{64} = 15.625\text{ Hz}, T=64\text{ us}$$

$$\frac{10 \times 10^{-3}}{64 \times 10^{-6}} = 156,25$$

No podemos tener en cuenta esos 25 decimales, y vamos a contar hasta 156. Entonces, en estas condiciones, para contar hasta 10 ms deberíamos dividir por 64 y contar hasta 156. Debemos además precargar el TMRO con 100.

Con este tipo de cuentas podemos llegar a contar tiempos de hasta 65 ms, si quiero contar más allá de eso debemos generar contadores adicionales, configurando registros adicionales.

¿Cómo hacemos entonces para conseguir valores superiores a éste, tales como 0,5s;1s;2s..?

- Esto se puede arreglar si tenemos un contador de mayor número de bits. La solución está en extender el timer0 con un registro (auxiliar) controlado por software.
- Dicho registro (auxiliar) contará el número de interrupciones por desbordamiento que genera el timer0, de forma de que este pase por cero, cuando haya pasado el tiempo que estamos calculando.

Ej:Siguiendo el ejemplo anterior, queremos ahora contar hasta 1s.

Debemos ver cuantas veces 10ms va a entrar en 1s, genero un contador aparte y lo hago por programa.

CONFIGURACIÓN DEL TMRO

Registro OPTION_REG

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
RBP	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

bit 7-RBP: Para habilitar Pull up interno del microcontrolador.

1=Inhabilito

0=Habilito

bit 6-INTEDG: Para seleccionar el flanco con el cual quiero que interrumpa, es más relevante cuando contamos eventos externos.

1=Flanco ascendente

0=Flanco descendente

bit 5-T0CS: Habilitar la transición en el pin de entrada o en la frecuencia interna

1=habilita el circuito oscilador interno del TMRO

0=habilita señal externa

bit 4- T0SE: Poco importante.

bit 3- PSA: Asigna prescaler al watchdog o al módulo del Timer

1=WDT

0=Timer

bit 2-0 - PSx: Selecciono por cuánto quiero que se divida.

Registro INTCON:Interrupt Control Register

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-x
GIE	PEIE	TOIE	INTE	RBIE ^(1,3)	TOIF ⁽²⁾	INTF	RBIF
bit 7							bit 0

bit 7-GIE: Habilita o deshabilita todas las interrupciones al mismo tiempo, conviene casi siempre arrancar con este pin en 0 y setearlo luego.

bit 6-PEIE: Habilidad de interrupción por periféricos.

bit 5-TOIE: Habilidad de interrupción por timer.

bit 4- INTE: Habilidad de interrupción por INT.

bit 3- RBIE: Habilidad de interrupción por PORTB

bit 2- TOIF: Flag de TMRO

Se pone en 1 cuando TMRO desborda, debo resetearla por programa

bit 1- INTIF: Flag de INT

bit 0- RBIF:Flag de PORTB

Cuando hay una interrupción, el micro pone en 0 el bit GIE para evitar que haya otra no deseada. Luego que finaliza la interrupción el bit GIE se pone en 1 (esto lo hace la instrucción RETFIE), pero a las banderas debo resetearlas yo.

SALVANDO EL ENTORNO

Esto se refiere a mantener el valor exacto que tienen los registros de trabajo STATUS y W, y cualquier otro de importancia, para que no sea modificado dentro de la subrutina de servicio de interrupción.

Para salvar el entorno cuando ocurre una interrupción se recomienda seguir los siguientes pasos:

- Almacenar a w en un registro general llamado W_TEMP
- Almacenar a STATUS en un registro general llamado STATUS_TEMP
- Se ejecuta la rutina de interrupción
- Restauramos a STATUS
- Restauramos a w

NOTA: Los registros generales, W_TEMP, STATUS_TEMP, y el de puertos deben ser declarados al inicio del programa, para no generar errores.

¿CUANDO UTILIZAR UNA INTERRUPCIÓN?

Una interrupción es muy importante cuando deseamos realizar una tarea que no dependa de una exploración constante del programa sobre la misma.

También cuando deseamos que nuestro programa responda de forma inmediata y automática a determinados eventos.

Ejemplo del uso de Interrupción:

- Cuando se necesita un pulsador de parada de emergencia (PARE RBO/INT).
- En el manejo de teclados. Ya que se puede meter al microcontrolador en un estado de reposo, ahorrando energía y esperando a que se pulse una tecla para iniciar una rutina de servicio.
- Cuando deseamos que el programa haga una tarea cada cierto tiempo, independientemente de lo que se encuentre haciendo en el programa principal. (DESBORDE DEL TMRO).

Para programar la interrupción por cambio de estado en los pines del puerto B en el microcontrolador, se necesita configurar correctamente el registro INTCON y el registro IOCB.

- **Paso 1:** Configurar los pines del Puerto B como entradas
Para que los pines del puerto B puedan detectar cambios, deben estar configurados como entradas. Esto se hace configurando los bits correspondientes en el registro TRISB.
`TRISB = 0xFF; // Configurar todos los pines del puerto B como entradas`
- **Paso 2:** Habilitar las interrupciones por cambio en el Puerto B
Las interrupciones por cambio de estado en el puerto B se controlan a través del bit RBIE en el registro INTCON.
`INTCONbits.RBIE = 1; // Habilita las interrupciones en cambio de estado en el puerto B`
- **Paso 3:** Seleccionar los pines que causarán la interrupción
Se debe especificar qué pines específicos del puerto B activarán la interrupción. Esto se hace en el registro IOCB. Cada bit en IOCB corresponde a un pin en el puerto B (RB7 a RB0).
`IOCB = 0x0F; // Habilita interrupción por cambio en los pines del Puerto B`
- **Paso 4:** Limpiar la bandera de interrupción del puerto B
Antes de habilitar las interrupciones globales, hay que asegurarse de que la bandera de interrupción de cambio en el puerto B (RBIF) está limpia para evitar una interrupción inmediata.
`INTCONbits.RBIF = 0; // Limpia la bandera de interrupción del puerto B`
- **Paso 5:** Habilitar las interrupciones globales
Finalmente, se debe habilitar las interrupciones globales para que el microcontrolador responda a las interrupciones.
`INTCONbits.PEIE = 1; // Habilita las interrupciones periféricas`
`INTCONbits.GIE = 1; // Habilita las interrupciones globales`

Manejo de la Interrupción

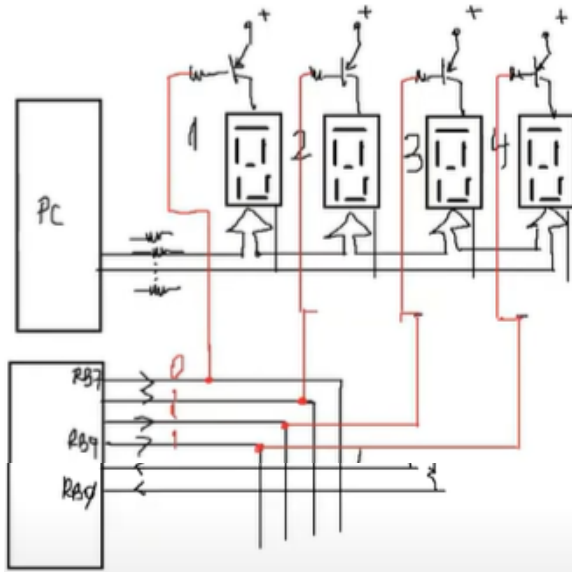
Aquí se debe implementar una rutina de servicio de interrupción (ISR) que maneje lo que sucede cuando se detecta un cambio en el estado de los pines del puerto B

DATAZO DEL TIMER 0

No puede despertar al PIC del sleep ya que no tiene un oscilador interno asociado que le permita encenderse cuando el PIC está en sleep

EJEMPLO: Supongamos el manejo de un teclado y display.

Mi sistema sería algo así:



El Puerto C va a estar definido como salida y el Puerto B mitad como salida y mitad como entrada. Esta es la forma más eficiente para manejar un display y un teclado con solo 2 puertos.

Lo habitual es usar 3 puertos.

Tenemos un sistema en el cual no solo vamos a sensor el teclado, sino que también vamos a encender los display

Con dos puertos podemos manejar hasta 4 display.

Entonces, estamos haciendo un sistema en el que estamos multiplexando 4 display y estamos censando al mismo tiempo un teclado de 16 bits

Tenemos que programar PC como salida y PB como E/S. Vamos a utilizar en este caso la interrupción

por TMRO para que me avise, por ejemplo, cada vez que tengo que mover las columnas.

- Precarga TMR0 5ms

$$\frac{1 \text{ MHz}}{32} = 31.250 \text{ Hz}, T=32 \text{ us}$$

$$\frac{5 \times 10^{-3}}{32 \times 10^{-6}} = 156,25$$

- Debemos entonces precargar TMR0 en 100, para realizar las 156 cuentas que necesito
- Asigno prescaler al TMR0 y lo hago dividir por 32 (100)

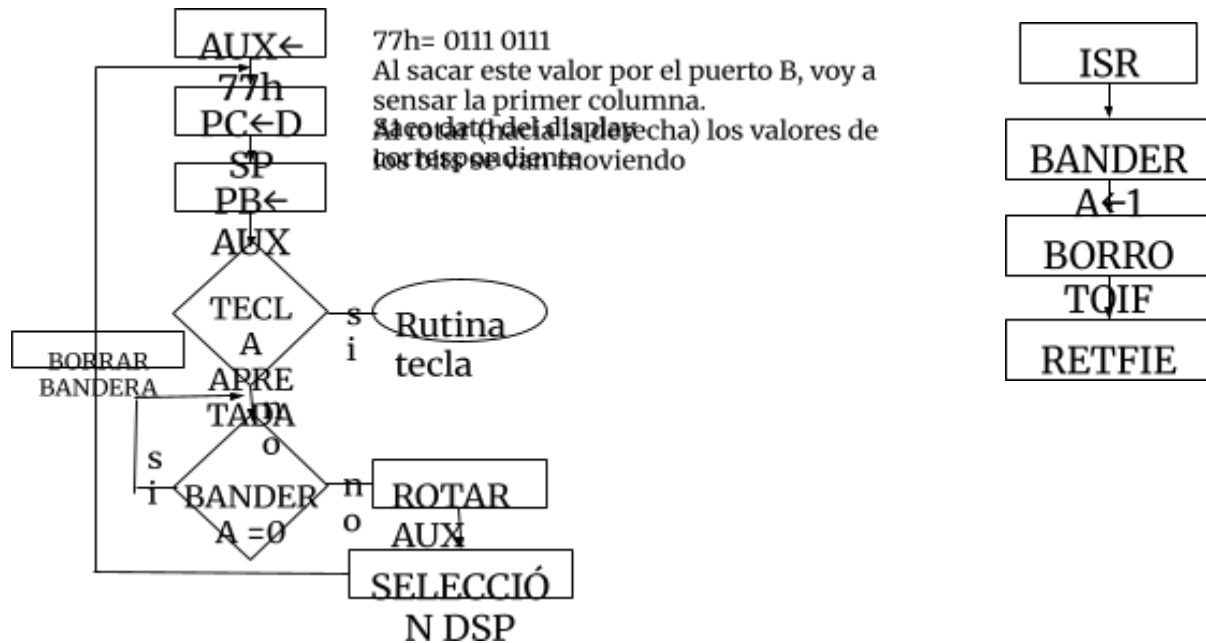
7							0
REGISTRO OPTION							
1	0	0	0	0	1	0	0

- Vamos a trabajar con una única interrupción, TMR0

7							0
REGISTRO INTCON							
1	0	1	0	0	0	0	0

Arrancó con GIE en 1 solo por cuestión de tiempo, pero por lo general se arranca con este bit en 0 y luego se setea.

PROGRAMA PRINCIPAL

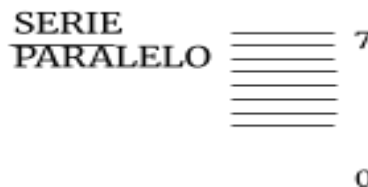


¿Por qué utilizamos 77h para encender?

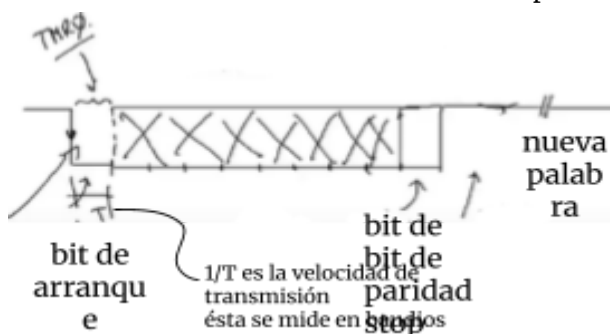
Este tipo de display se enciende por '0' y poniendo '1' en todos los demás, de esta manera encendemos uno por vez. En la siguiente interrupción esto cambia, al rotar el número, el bit en '0' va a encenderme el siguiente display y así sucesivamente.

COMUNICACIÓN

Hay dos formas para comunicarse entre el microprocesador y el mundo exterior, estas son en serie y en paralelo. Éste último ya casi ni se usa.



Para comunicación en serie debo tener un protocolo, ya que voy enviando los bits uno detrás de otro.



El protocolo que se utiliza es el RS-232, este protocolo es más o menos así:

Yo arranco en '1', hago un cambio a '0' y ahí hay lo que se llama un **'bit de arranque'**. Este protocolo es asíncrono, no hay un reloj común sino que se sincroniza con este flanco. Tanto transmisor como receptor saben cual es la frecuencia a la cual yo estoy transmitiendo. A continuación de este bit vienen 8 bits, todos demoran el mismo tiempo. Luego puede venir un bit de paridad y uno o dos bits de stop, y ahí puede arrancar la nueva palabra.

Este es el protocolo, todo viene por **un solo bit**. Puedo hacer que el TMRO funcione como un reloj de transmisión 0, entonces genero estos tiempos de forma muy precisa con un cristal.

El **bit de paridad** lo que hace es, si yo trabajo con paridad par, y la cantidad de 1 que hay en mi número es par, el bit de paridad se pone en 0, en caso contrario se pondrá en 1 para que mi número sea par. Si se trabaja con paridad impar sería lo contrario.

MÓDULOS ANALÓGICOS

Parte de los puertos digitales se comparten con los módulos analógicos, es decir pueden estar ambos en un mismo pin.

NUCLEO CPU + MEMORIAS

ALU, sincronización y control, (puertos E/S, ADC, PC, clock, etc.)

PERIFERICOS

Continuando con el análisis de la arquitectura del PIC16F887 seguimos con el MODULOS ANALOGICOS

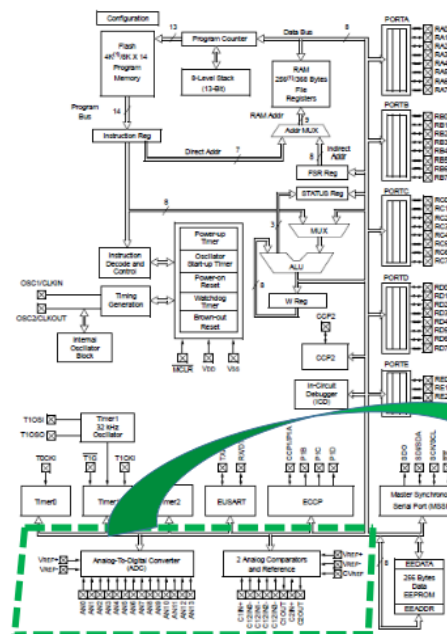
PERIFERICOS

(puertos E/S, ADC, temporizadores, comparadores, PWM, Transmisión serie)

Vamos a ver la parte del ADC, no la parte de los comparadores

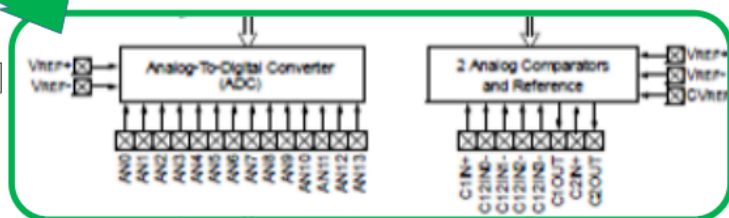
¿Pero para qué me sirve a mi el módulo convertor?

Tenemos 14 entradas para hacer ingresar señales analógicas, no 14 módulos ADC. Esto solo nos permite ingresar una sola señal a la vez



MODULOS ANALÓGICOS

- ✓ 1 Conversor Analógico/Digital - ADC
- ✓ 2 Comparadores Analógicos – C1 y C2



¿Cuáles son los parámetros que determinan un ADC?

En primer lugar, el tipo de ADC (como va a hacer la conversión del dominio analógico al dominio digital), y después algo que es muy importante es la cantidad de bits (en cuántos niveles me va a convertir esa señal analógica en digital).

En el caso del PIC16F887 tenemos un ADC de 10 bits, y es de aproximaciones sucesivas.

En este caso no es relevante que el ADC sea de aproximaciones sucesivas, es decir no nos interesa su arquitectura, ya que esta nos define la velocidad de trabajo, potencia que consume. Lo que a nosotros nos importa es que convierta una cierta cantidad de bits (en este caso 10 bits)

Funcionamiento de ADC de aproximaciones sucesivas:

El ADC toma un dato, lo guarda en sample and hold, y luego toma ese dato y empieza con comparaciones. Por ejemplo, “¿es mayor que la mitad del rango?” “sí”, “¿es mayor que $\frac{3}{4}$ del rango?” “no” y así.. Va por aproximaciones sucesivas llegando al valor más cercano.

Características:

Convierte el valor analógico en valor binario de 10 bits - conversor de aproximaciones sucesivas y lo guarda en ADRESH y ADRESL

Las entradas son multiplexadas a un circuito único de sample and hold, esto significa que podemos hacer una sola conversión por unidad de tiempo de conversión, una vez finalizada ésta recién ahí se puede pasar a convertir otra señal en otro pin. Pero no se puede hacer al mismo tiempo, o sea no puedo meter 14 señales a la vez y querer esas 14 conversiones en ese momento. Además, esas 14 entradas analógicas nos quitan entradas digitales

Dispone de 14 entradas analógicas independientes

El voltaje de referencia puede seleccionarse por software, es decir, no solo trabajar con 0 y vcc, sino que podemos poner distintos voltajes de referencia y aumentar el rango dinámico. Pero para esto vamos a tener que consumir 2 de los 14 pines analógicos.

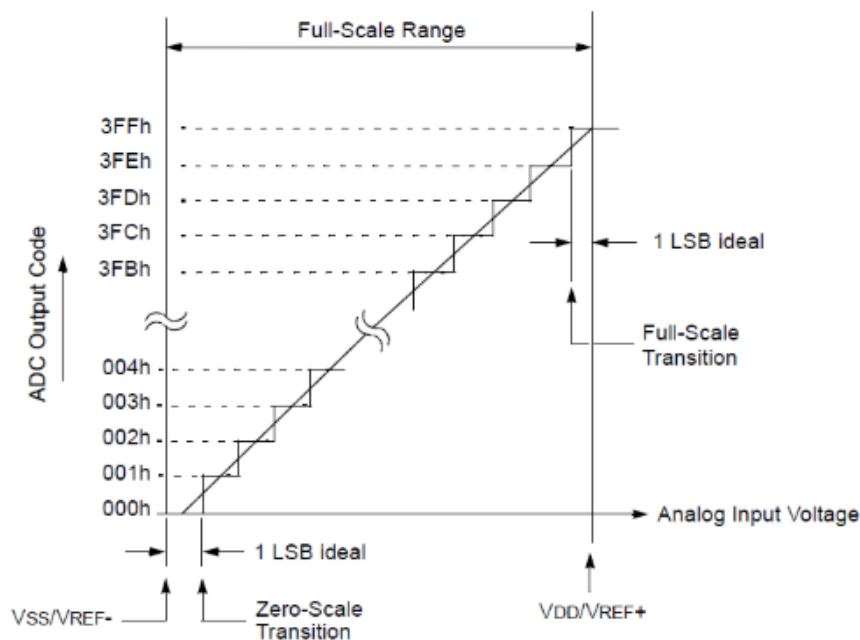
Al finalizar la conversión puede generar una interrupción, esto resulta en una ventaja porque podemos dejar al ADC funcionando, y cuando termina nos avisa, y mientras yo puedo ir haciendo otras cosas en el medio.

Puede generar interrupción para wake-up en Sleep. En este estado todo el PIC va a estar en bajo consumo, no el ADC, a éste lo podemos poner a convertir y cuando finalice la conversión genera una interrupción que hace que el PIC se despierte y vaya a la posición 04h

Curva típica del ADC

Tenemos una tensión de referencia, y esta curva va desde -VREF hasta +VREF (eje x)

En el eje y está la salida del ADC, el código binario que vamos a tener en el ADC. El máximo valor va a estar en 3FFh y el mínimo valor va a estar en 000h



CIRCUITO ADC

Se observa claramente que hay **un solo ADC**, y tenemos además un **multiplexor** que va eligiendo entre las entradas. Dentro de éstas, están las tensiones de referencia.

CVREF son las tensiones de referencia por default y $\pm VREF$ son las configurables, para poder elegir las vamos a tener que utilizar el PIN AN2 y el PIN AN3.

Otro bit interesante es el bit **GO/DONE**, su función es decirle al ADC que empiece a convertir (GO), y cuando termine de convertir se pone en 0 este bit (DONE) y además genera el flag de la interrupción.

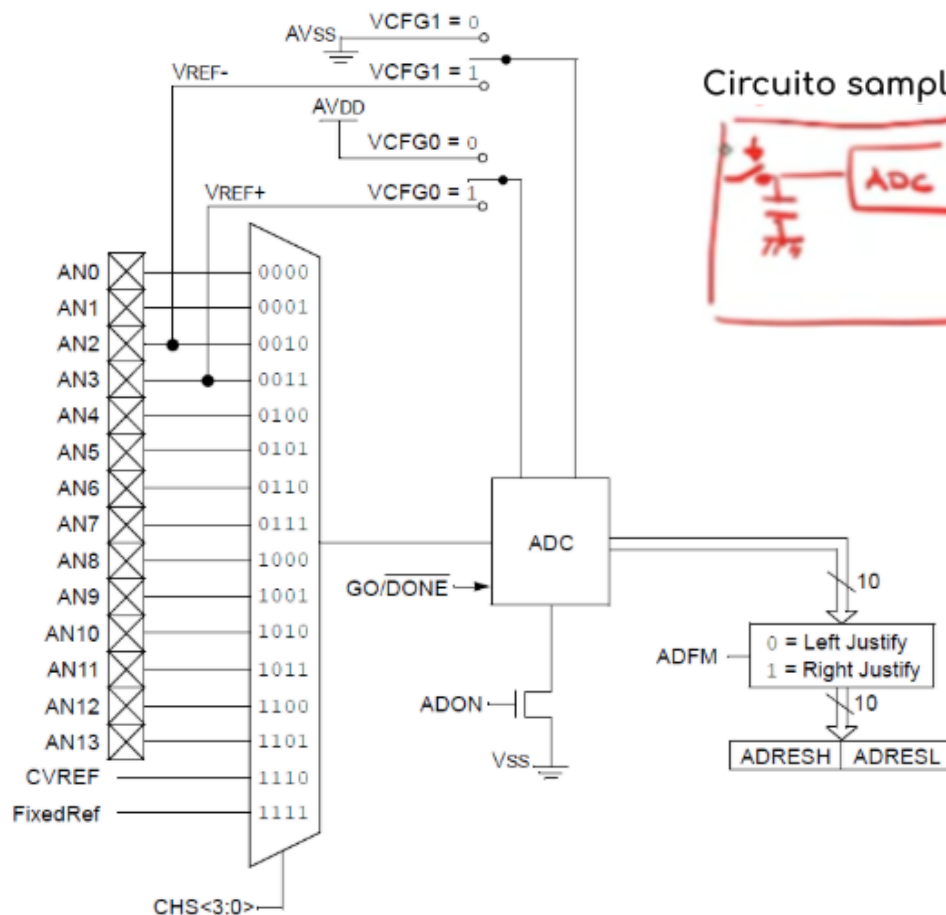
Otra cosa a tener en cuenta es que debemos encender el ADC antes de darle GO y esperar cierto tiempo, ésto se hace con el bit **ADON** al excitar la base del transistor.

El bit **ADFM** es el **formato**. El ADC tiene **10 bits en memoria**, sabemos que el ancho de memoria del PIC es de 8 bits, por lo que si yo quiero grabar lo que sale del ADC tengo que grabarlo en 2 bytes.

De este bit depende como los grabo:

- **Justificación a la derecha (ADFM=1):** Los bits menos significativos se colocan en un byte y los dos bits restantes se colocan en otro byte.
- **Justificación a la izquierda (ADFM=0):** Los 8 bits más significativos se colocan en un byte y los dos menos significativos se colocan en otro byte.

Los bits se guardan en ADRESH y ADRESL.



CONFIGURACIÓN DEL ADC

1. CONFIGURACIÓN DE PUERTOS

Para conversión analógica debe colocarse la entrada del puerto en entrada analógica

2. SELECCIÓN DEL CANAL

Los bits de selección CHS en el Registro ADCON0 determinan qué canal/entrada será multiplexado al circuito sample-and-hold.

Cuando se cambia de canal se requiere un retardo antes de iniciar la nueva conversión (2TAD).

ADCON0 <5:2>, bits CHS<3:0>

3. SELECCIÓN DEL VOLTAJE DE REFERENCIA

Los bits VCFG del ADCON1 permiten seleccionar Voltaje de Referencia, al hacer esto ya condicionamos el uso del PIN AN2 y el PIN AN 3. No es posible cambiar el voltaje de referencia mientras se está convirtiendo

Referencia de Voltaje positivo: VDD o de fuente externa

Referencia de Voltaje negativo: Vss o de fuente externa

ADCON1 <5:4>, bits VCFG<1:0>

4. SELECCIÓN DEL OSCILADOR PARA LA CONVERSIÓN

El ADC no hace una conversión instantánea, ya que es una maquinita de estado que necesita un clock y va convirtiendo de un clock a la vez.

Entonces debemos darle un clock para que trabaje y dependiendo del clock que yo le de va a depender la velocidad de conversión y la potencia que consume.

El oscilador que se usa en la conversión puede seleccionarse por software.

ADCON0 <7:6>, bits ADCS<1:0>

TABLE 9-1: ADC CLOCK PERIOD (TAD) Vs. DEVICE OPERATING FREQUENCIES (VDD ≥ 3.0V)

ADC Clock Period (TAD)		Device Frequency (Fosc)			
ADC Clock Source	ADCS<1:0>	20 MHz	8 MHz	4 MHz	1 MHz
FOSC/2	00	100 ns ⁽²⁾	250 ns ⁽²⁾	500 ns ⁽²⁾	2.0 μs
FOSC/8	01	400 ns ⁽²⁾	1.0 μs ⁽²⁾	2.0 μs	8.0 μs ⁽³⁾
FOSC/32	10	1.6 μs	4.0 μs	8.0 μs ⁽³⁾	32.0 μs ⁽³⁾
FRC	11	2-6 μs ^(1,4)	2-6 μs ^(1,4)	2-6 μs ^(1,4)	2-6 μs ^(1,4)

FRC es un clock interno que es mucho más lento, pero nos permite despertar al PIC del sleep.

Lo que tiene de raro es que no tiene un valor fijo, tiene una varianza que va de 2 a 6. Nunca vamos a saber con qué frecuencia va a trabajar, esto depende del proceso con el que fue hecho el PIC en ese momento.

Los valores en gris no son admisibles, no podemos utilizarlos. Entonces por más que yo tenga una frecuencia de 20MHz estoy limitado a trabajar con un clock de FOSC/32.

Esto es porque estos valores en grises violan el tiempo mínimo requerido para la adquisición del dato (tiempo que le toma al sample and hold tomar el dato y dejarlo quieto para que empiece a funcionar el ADC). **Recordar que siempre hace falta que uno tome el dato, lo deje quieto, y durante todo el tiempo que esté haciendo la conversión ese dato quede estable, esto lo hace el circuito sample and hold. Si no estuviera este circuito los valores estarían cambiando todo el tiempo y no podríamos convertir nunca lo que queremos.**

Entonces, para poder hacer funcionar el ADC no importa la cantidad de MHZ que tengamos, lo que tenemos que fijarnos es cuales son los clocks que no violan el tiempo de adquisición.

Note 1: The FRC source has a typical TAD time of 4 μs for VDD > 3.0V.

2: These values violate the minimum required TAD time.

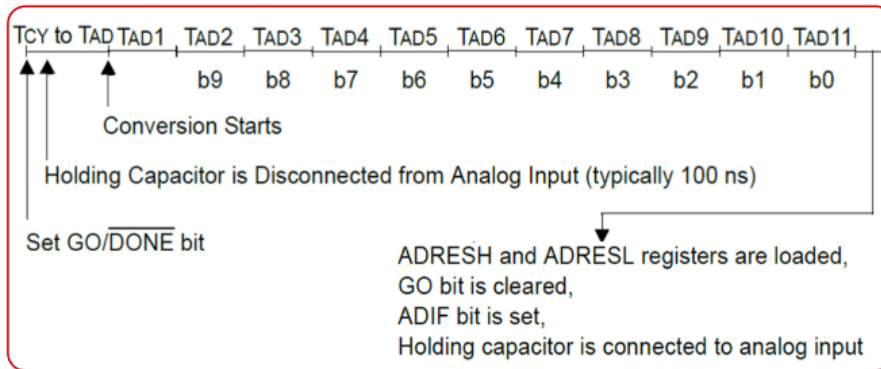
3: For faster conversion times, the selection of another clock source is recommended.

4: When the device frequency is greater than 1 MHz, the FRC clock source is only recommended if the conversion will be performed during Sleep.

TIEMPO DE LA CONVERSIÓN DEL ADC

El tiempo para completar la conversión de 1 bit se define como TAD.

El tiempo total para convertir los 10 bits es de 11 TAD



Una vez que yo seteo el **GO/DONE** empieza mi conversión, a partir de ahí tengo que esperar un tiempo hasta que el capacitor toma un dato y se desconecta de la entrada analógica, quedando el dato estable (TCY-TAD)

En el TAD1 aún no comenzó a convertir porque la comparación la hace después de TAD1 y recién ahí tiene b9. La conversión comienza en el TAD2. **Siempre empieza a convertir desde el bit más significativo**, ya que este bit me divide el rango de 50 para arriba 50 para abajo, el bit siguiente va a dividir una de estas mitades en cuatro cuadrantes y así hasta llegar al bit 0.

Una vez que esto termina va a guardar en el **ADRESH** y **ADRESL** los datos, luego, además de eso va a bajar el bit de **GO/DONE**, y si la interrupción está habilitada va a subir la bandera del AD. Una vez que hizo todo esto va a volver a conectar el capacitor del circuito sample and hold a la entrada analógica.

TIEMPO DE ADQUISICIÓN:

- Tiempo entre seleccionar una entrada analógica y el inicio de la conversión propiamente dicha.
- Depende de la impedancia de la fuente (máxima impedancia recomendada 10kΩ).
- El valor típico es de 11,50 μs
- Antes de empezar otra conversión se requiere un delay de 2TAD

TABLE 17-11: PIC16F882/883/884/886/887 A/D CONVERSION REQUIREMENTS

Standard Operating Conditions (unless otherwise stated) Operating temperature $-40^{\circ}\text{C} \leq T_A \leq +125^{\circ}\text{C}$							
Param No.	Sym.	Characteristic	Min.	Typ†	Max.	Units	Conditions
AD130*	TAD	A/D Clock Period	1.6	—	9.0	μs	TOSC-based, $V_{\text{REF}} \geq 3.0\text{V}$
			3.0	—	9.0	μs	TOSC-based, V_{REF} full range
		A/D Internal RC Oscillator Period	3.0	6.0	9.0	μs	ADCS<1:0> = 11 (ADRC mode)
			1.6	4.0	6.0	μs	At $V_{\text{DD}} = 5.0\text{V}$
AD131	TCNV	Conversion Time (not including Acquisition Time) ⁽¹⁾	—	11	—	TAD	Set GO/DONE bit to new data in A/D Result register
AD132*	TACQ	Acquisition Time	—	11.5	—	μs	
AD133*	TAMP	Amplifier Settling Time	—	—	5	μs	
AD134	TGO	Q4 to A/D Clock Start	—	Tosc/2	—	—	If the A/D clock source is selected as RC, a time of Tcy is added before the A/D clock starts. This allows the SLEEP instruction to be executed.
			—	Tosc/2 + Tcy	—	—	

* These parameters are characterized but not tested.

† Data in "Typ" column is at 5.0V, 25°C unless otherwise stated. These parameters are for design guidance only and are not tested.

Note 1: ADRESH and ADRESL registers may be read on the following Tcy cycle.

2: See Section 9.3 "A/D Acquisition Requirements" for minimum conditions.

1. CONTROL DE INTERRUPCIÓN

Al finalizar la conversión pone en 1 el flag ADIF siempre.

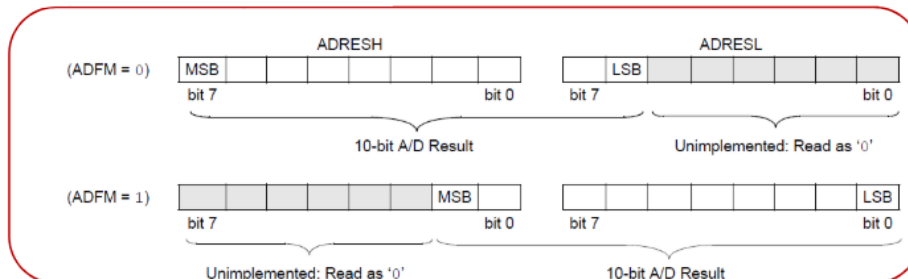
Si está funcionando por interrupción irá a 0004H si:

- **GIE= 1 y PEIE= 1** (Registro INTCON)
- **ADIE = 1** (Registro PIE1)

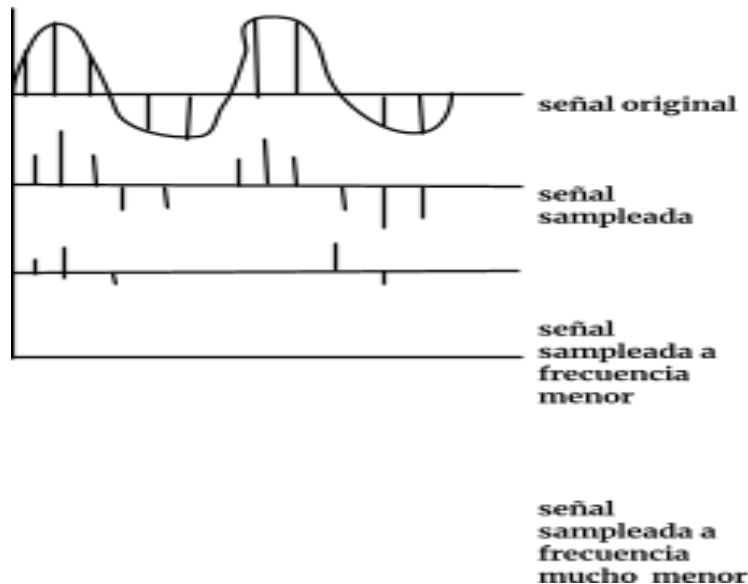
El flag ADIF (Registro PIR1) debe ser puesto a 0 Puede convertir en modo SLEEP utilizando como oscilador FRC

2. TIPO DE FORMATO DEL RESULTADO DIGITAL

El resultado puede ser entregado en dos formatos: justificado a la derecha o justificado a la izquierda. El bit 7 - ADFM del ADCON1 permite seleccionar el formato.



¿Cuál va a ser la diferencia si yo cambio el clock de muestreo?



Debemos cumplir con el **Teorema de Muestreo**.

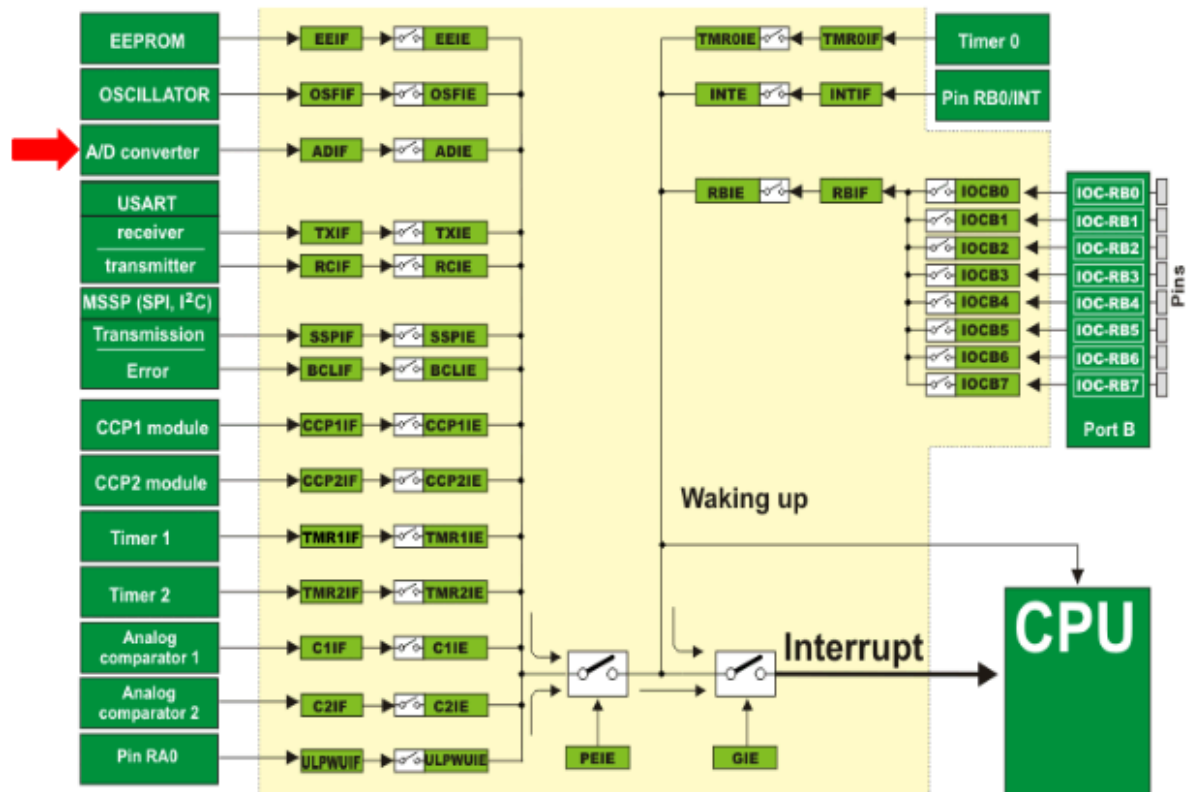
Para poder recuperar mi señal la $f_m = 2F$ donde F es la frecuencia de la señal a muestrear.

Por más que estemos cumpliendo Nyquist debemos ser cuidadosos si f_m es exactamente $2F$

INTERRUPCIÓN CON ADC

Si yo quiero habilitar la interrupción del conversor lo que tengo que hacer es habilitar el **ADIE**, después debo habilitar el **PEIE** y el **GIE**. Y esto recién va a generar una interrupción en la CPU

INTCON	GIE	PEIE	TOIE	INTE	RBIE ⁽¹⁾	TOIF ⁽²⁾	INTF	RBIF
PIE1	—	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF



EL ADC EN FUNCIONAMIENTO

¿Qué es lo que necesito setear para que empiece a funcionar mi ADC?

Lo primero que tengo que hacer es habilitar la conversión del ADC, para esto debemos encenderlo, a esto lo hacemos colocando un 1 en el bit **ADON** del registro **ADCON0**.

Lo próximo que se debe hacer es encender el bit **GO/DONE** colocando un 1

Entonces, con el bit **ADON** encendemos el ADC, y con el **GO/DONE** inicia a convertir

Registro de Control ADC 0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
ADCON0	ADCS1	ADCS0	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON
bit 7							1	1 bit 0

estos bits no se pueden poner a 1 en la misma instrucción

Fin de la conversión

- El ADC coloca un 0 en el bit **GO/DONE**
- Coloca un 1 en el bit de flag de interrupción **ADIF**
- Actualiza los registros **ADRESH:ADRESL** con el resultado de la conversión

Registro de Control ADC 0:	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
ADCON0	ADCS1	ADCS0	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON
bit 7							0	1 bit 0

Registro Requiere Interrupción 1:	U-0	R/W-0	R-0	R-0	R/W-0	R/W-0	R/W-0	R/W-0
PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
bit 7		1						bit 0

Registros Resultado Conversión:	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
ADRESH-ADRESL	ADRES9	ADRES8	ADRES7	ADRES6	ADRES5	ADRES4	ADRES3	ADRES2
(justificado a la izquierda ADFM=0)	ADRES1	ADRES0	—	—	—	—	—	—
bit 7								bit 0

Abortar la Conversión:

- Para terminar la conversión antes que finalice la misma, se coloca el bit **GO/DONE** en 0
- Los registros **ADRESH:ADRESL** no se actualizan y muestran el valor de la conversión anterior.
- Antes de iniciar cualquier otra adquisición se debe hacer un retardo de **2 TAD delay**, porque al no permitir que la conversión haya finalizado, el ADC no volvió a conectar al capacitor.
- La nueva conversión iniciará en el canal que está programado, es decir, no necesito volver a programar el canal, pues no se pierde la programación

Nota: Un reinicio del dispositivo fuerza a todos los registros a su estado de reinicio. Esto significa que el módulo ADC se apaga y se termina cualquier conversión pendiente de ser terminada y que el módulo ADC no podrá convertir señales analógicas hasta que se haya inicializado por completo.

Operando durante el SLEEP

- Debe operar con el **oscilador interno FRC**, si uso otro no se va a del sleep.
- Si se selecciona FRC el conversor **espera una instrucción para iniciar la conversión**, esto es para poder darle tiempo a poder prender el SLEEP.
- Esto permite que la instrucción SLEEP se ejecute y reduce el ruido
- Si la interrupción está habilitada despertará al micro al setearse ADIF
- Si la interrupción está deshabilitada el ADC se apagará al terminar la conversión pero ADON seguirá en 1.
- Si el oscilador no es FRC el ADC se aborta la la conversión pero ADON seguirá en 1.

REGISTROS DEL ADC

Registro de Control ADC 0: ADCON0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
	ADCS1	ADCS0	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON
	bit 7							bit 0

Este registro tiene la selección del clock, la selección del canal y los bits GO/DONE y ADON

bit 7-6 **ADCS<1:0>**: A/D Conversion Clock Select bits

00 = Fosc/2
01 = Fosc/8
10 = Fosc/32
11 = FRC (clock derived from a dedicated internal oscillator = 500 kHz max)

bit 5-2 **CHS<3:0>**: Analog Channel Select bits

0000 = AN0
0001 = AN1
0010 = AN2
0011 = AN3
0100 = AN4
0101 = AN5
0110 = AN6
0111 = AN7
1000 = AN8
1001 = AN9
1010 = AN10
1011 = AN11
1100 = AN12
1101 = AN13
1110 = CVREF
1111 = Fixed Ref (0.6V Fixed Voltage Reference)

bit 1 **GO/DONE**: A/D Conversion Status bit

1 = A/D conversion cycle in progress. Setting this bit starts an A/D conversion cycle.
This bit is automatically cleared by hardware when the A/D conversion has completed
0 = A/D conversion completed/not in progress

bit 0 **ADON**: ADC Enable bit

1 = ADC is enabled
0 = ADC is disabled and consumes no operating current

Registro de Control ADC 1: ADCON1	R/W-0	U-0	R/W-0	R/W-0	U-0	U-0	U-0	U-0
	ADFM	—	VCFG1	VCFG0	—	—	—	—
	bit 7							bit 0

Este registro tiene el bit de formato (ADFM), los dos bits de tensión de referencia (VCFG1 y VCFG0)

bit 7 **ADFM**: A/D Conversion Result Format Select bit

1 = Right justified
0 = Left justified

bit 6 **Unimplemented**: Read as '0'

bit 5 **VCFG1**: Voltage Reference bit

1 = VREF- pin
0 = Vss

bit 4 **VCFG0**: Voltage Reference bit

1 = VREF+ pin
0 = VDD

bit 3-0 **Unimplemented**: Read as '0'

TABLE 9-2: SUMMARY OF ASSOCIATED ADC REGISTERS

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Register on Page
ADCON0	ADCS1	ADCS0	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON	104
ADCON1	ADFM	—	VCFG1	VCFG0	—	—	—	—	105
ANSEL	ANS7	ANS6	ANS5	ANS4	ANS3	ANS2	ANS1	ANS0	41
ANSELH	—	—	ANS13	ANS12	ANS11	ANS10	ANS9	ANS8	49
ADRESH	A/D Result Register High Byte								106
ADRESL	A/D Result Register Low Byte								106
INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	32
PIE1	—	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	33
PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	35
PORTA	RA7	RA6	RA5	RA4	RA3	RA2	RA1	RA0	40
PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0	49
PORTE	—	—	—	—	RE3	RE2	RE1	RE0	60
TRISA	TRISA7	TRISA6	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	40
TRISB	TRISB7	TRISB6	TRISB5	TRISB4	TRISB3	TRISB2	TRISB1	TRISB0	49
TRISE	—	—	—	—	TRISE3	TRISE2	TRISE1	TRISE0	60

Legend: x = unknown, u = unchanged, — = unimplemented read as '0'. Shaded cells are not used for ADC module.

EJEMPLO DE PROCEDIMIENTO DE CONVERSIÓN

- Configure Port:
 - Disable pin output driver (See TRIS register)
 - Configure pin as analog
- Configure the ADC module:
 - Select ADC conversion clock
 - Configure voltage reference
 - Select ADC input channel
 - Select result format
 - Turn on ADC module
- Configure ADC interrupt (optional):
 - Clear ADC interrupt flag
 - Enable ADC interrupt
 - Enable peripheral interrupt
 - Enable global interrupt⁽¹⁾
- Wait the required acquisition time⁽²⁾.
- Start conversion by setting the GO/DONE bit.
- Wait for ADC conversion to complete by one of the following:
 - Polling the GO/DONE bit
 - Waiting for the ADC interrupt (interrupts enabled)
- Read ADC Result
- Clear the ADC interrupt flag (required if interrupt is enabled).

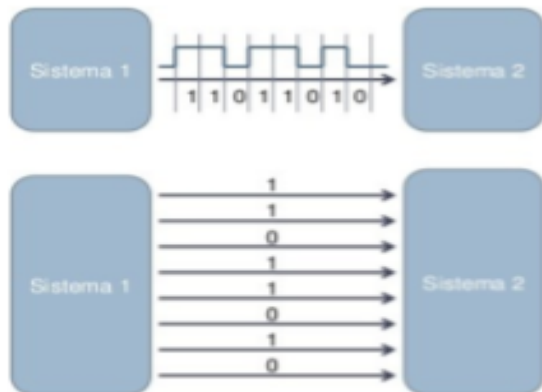
```

;This code block configures the ADC
;for polling, Vdd and Vss as reference, Frc
clock and AN0 input.
;
;Conversion start & polling for completion
; are included.
;
BANKSEL    ADCON1    ;
MOVLW     B'10000000' ;right justify
MOVWF     ADCON1     ;Vdd and Vss as Vref
BANKSEL    TRISA     ;
BSF       TRISA,0    ;Set RA0 to input
BANKSEL    ANSEL     ;
BSF       ANSEL,0    ;Set RA0 to analog
BANKSEL    ADCON0    ;
MOVLW     B'11000001' ;ADC Frc clock,
MOVWF     ADCON0     ;AN0, On
CALL      SampleTime ;Acquisition delay
BSF       ADCON0,GO   ;Start conversion
BTFSC     ADCON0,GO   ;Is conversion done?
GOTO      $-1        ;No, test again
BANKSEL    ADRESH    ;
MOVWF     ADRESH,W   ;Read upper 2 bits
MOVWF     RESULTHI   ;store in GPR space
BANKSEL    ADRESL    ;
MOVWF     ADRESL,W   ;Read lower 8 bits
MOVWF     RESULTLO   ;Store in GPR space

```


COMUNICACIÓN SERIE

COMUNICACIÓN SERIE VS PARALELA



SERIE

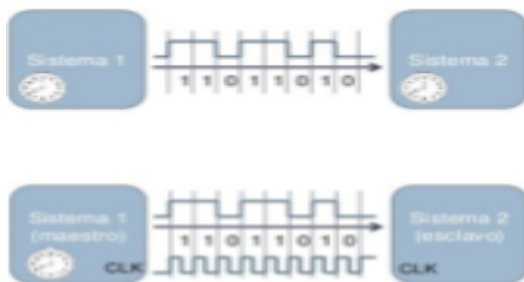
los bits se transmiten en serie, uno detrás de otro, se van desplazando.
sólo se necesita un hilo.
es más lento
necesita un protocolo, handshaking

PARALELO

No se utiliza mucho, los dispositivos deben estar muy cerca y necesito muchos cables

todos los bits se transmiten a la vez
se necesitan 8 hilos
más rápido

COMUNICACIÓN SERIE SÍNCRONA VS ASÍNCRONA



ASÍNCRONA

Cada sistema tiene su propio reloj local, no hay un reloj que sincroniza los bits
los clocks deben tener la misma frecuencia
sincronización mediante bit de Start y Stop (para saber dónde termina)
solo 1 hilo, debemos idear algún tipo de sincronización

SÍNCRONA

una señal de reloj común
el maestro genera el reloj, todo funciona en base a este clock, no necesita start y stop
dos hilos, por uno va el dato y por el otro va el clock, por lo que no debo hacer sincronización
velocidades mayores

COMUNICACIÓN SERIE FULL-DUPLEX VS HALF-DUPLEX



FULL-DUPLEX

Comunicación bidireccional simultánea
Dos canales de datos (TX,RX)
Dos hilos

HALF-DUPLEX

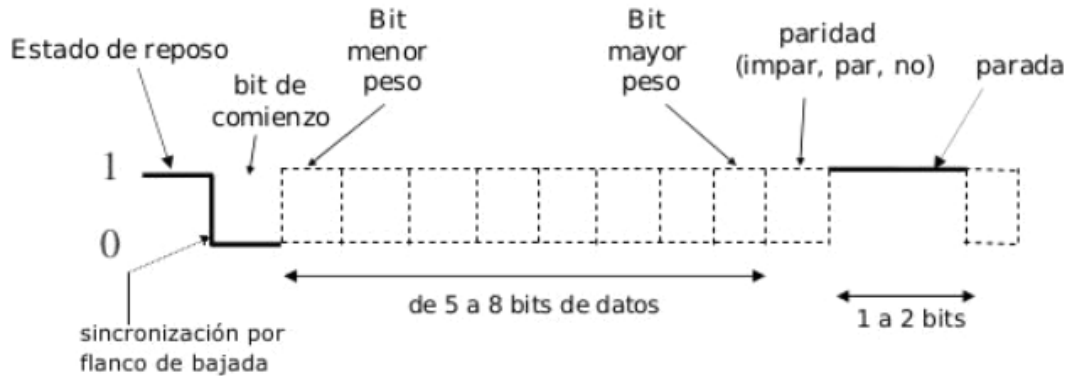
Comunicación bidireccional multiplexada en el tiempo
Un único canal (DATA)
Primero se transmite en un sentido, luego en el otro (protocolo), no podemos tener transmisión simultánea
Un hilo

PROTOCOLO CAPA FÍSICA-Serie RS232

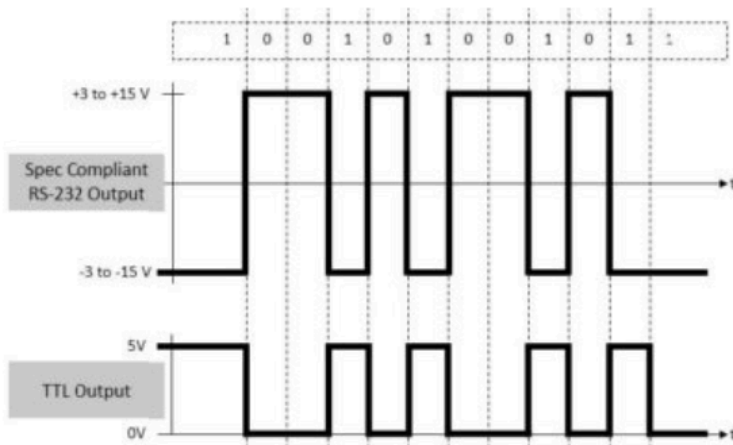
Tiene que ver con como yo transmito el dato por el canal.

Una cosa es mi protocolo de capa digital que es la que yo trabajo dentro del chip, que van a ser 0 y 1, mientras que cuando yo lo saque afuera ahí lo que voy a hacer es poner algún tipo de modulación, algo que me permita adecuarme al canal al cual yo estoy transmitiendo los datos.

La comunicación serie RS232 proporciona una forma estándar y confiable de transferir datos digitales entre dispositivos, utilizando cables de conexión y puertos serie en los dispositivos involucrados.



Lo que a nosotros nos importa ahora es que vamos a transmitir fuera del micro utilizando esto. Del micro va a salir como en el gráfico de TTL Output.



Vamos a tener que conectarlo a algo para comunicarnos hacia afuera. Si yo me comunico de un PIC a otro PIC no me hace falta poner nada porque los dos entienden lo mismo, pero si yo me comunico de un PIC a la computadora ahí ya no voy a poder utilizar eso. Y además vamos a tener que cambiar de protocolo, debido a que las máquinas ya no utilizan el puerto serie RS232, ahora utilizan el USB.

MÓDULO COMUNICACIÓN SERIE

- Transmisión y recepción full-duplex asíncrona, puedo transmitir hacia la computadora y la computadora me va a poder mandar datos al mismo tiempo.
- Tiene un input buffer de dos caracteres
- Un output buffer de un carácter
- Podemos programarlo para transmitir 8 o 9 bits
- Nos permite detectar si hubo un overflow de datos, si estamos escribiendo arriba del receptor más veces de las que hemos leído
- Nos permite ver si hay un problema en el framing, osea si el dato que nos llegó no cumple con el protocolo
- También podemos programar la polaridad del clock, lo vamos a usar en modo síncrono
- Podemos trabajar en modo sleep.

MODULO EUSART- Enhanced Universal Synchronous Asynchronous Receiver Transmitter

Nosotros no estamos mandando la señal de clock de un lado a otro, no podemos hacerlo ya que estamos mandando solamente los datos.

¿Cómo hago para saber cuándo empieza y cuándo termina un dato?

Para esto hay que establecer un protocolo, se dice protocolo de handshaking, y los dos lo deben conocer. Es importante que ambos usen el mismo protocolo y la misma frecuencia de clock.

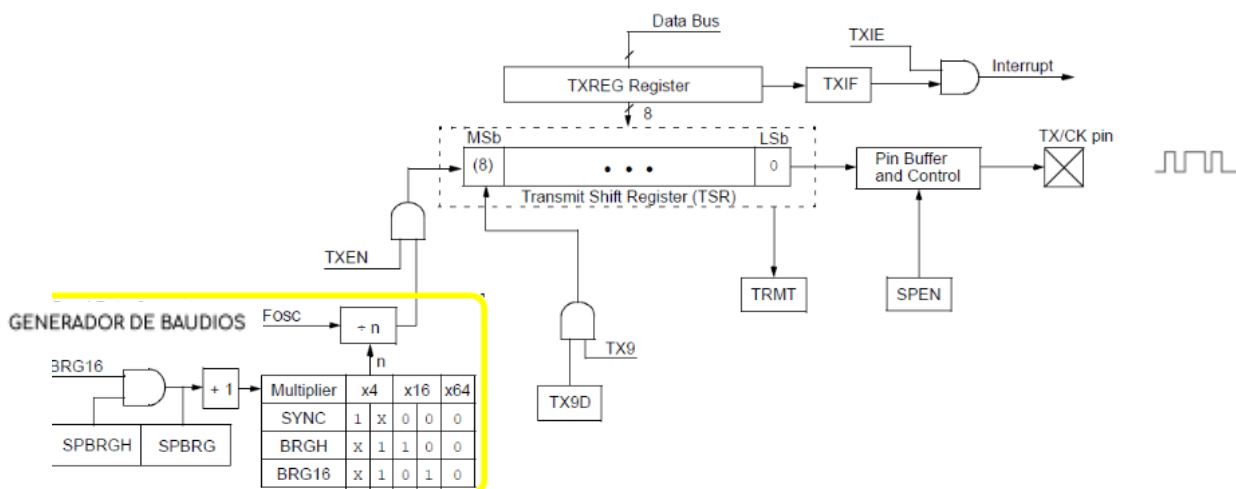
Modo Asincrónico

Lo primero que vamos a decir es cual es el **estado inactivo del canal**, si yo siempre tengo un 1 en el canal no estoy transmitiendo nada. Cuando yo empiezo a transmitir lo que yo voy a decir es “durante un tiempo de bit voy a poner un 0”, y este 0 es el **bit de estado**. Entonces lo que va a estar viendo el micro es que haya una transición de 1 a 0. El próximo bit que viene va a ser mi **bit menos significativo**, después el 2,3,...8/9. Una vez que terminó de transmitir, va a venir el **bit de parada**, el cual va a ser 1.

Para transmitir un byte se procede así:

Cuando no transmite, la línea de datos permanece en estado alto (1)
 La transmisión de un byte comienza con un bit de arranque (START) que es cero (0)
 El dato tiene un ancho de 8 o 9 bits (primero se transmite el bit menos significativo - LSB)
 La transmisión de un byte termina con bit de parada (STOP) que es uno (1)

Con esta información armamos un circuito, tanto para transmitir como para recibir.

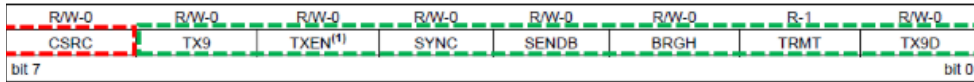
EUSART Modo Transmisor**¿Qué necesito para transmitir en serie?**

Una **frecuencia del clock**, algo que me vaya generando un bit por unidad de tiempo que yo le diga. Esto nos va a dar a cuantos baudios (bits por segundo) estoy transmitiendo. El circuito en amarillo es el **generador de baudios**. Yo entro con una frecuencia de oscilación y dependiendo como estén configurados algunos registros a esa frecuencia del clock la voy a dividir, y con esta frecuencia de clock y dependiendo si está habilitado mi transmisor (TXEN) voy a empezar a mandar los datos que tengo en el registro TSR, a este registro lo vamos a cargar con el registro **TXREG Register**, al cual tenemos acceso, y lo que hacemos es guardar datos que queremos transmitir en el TXREG y los paso al registro auxiliar TSR. Y con el clock voy a ir sacando de un bit a la vez haciendo un shift register. Además si yo tengo habilitado la transmisión de un noveno bit (TX9D) vamos a transmitir este dato.

Otra cosa interesante es que cuando pasan los datos de TXREG pasa al TSR se **habilita la interrupción**.
¿Por qué no se genera la interrupción cuando yo terminé de convertir?

Porque lo importante acá para el micro, es que yo pueda poner un dato nuevo dentro del registro al que yo tengo acceso, y yo a esto puedo hacerlo una vez que termine de enviar los datos del TXREG al TSR, y es por esto que aquí se genera la interrupción, siempre y cuando yo tenga la bandera habilitada. La interrupción se generaría antes de transmitir, yo lo que necesito es poder tener acceso al TXREG, la transmisión la realiza otro circuito.

TXSTA: Registro de Estado y Control de Transmisión



Sincrónico
Sinc/Asinc

Registros para configurar el transmisor asíncrono

- **TX9:** Habilidad de transmisión del bit 9
- **TXEN:** Habilidad del transmisor/clock
 - 1=Modo maestro
 - 0=Modo esclavo
- **SYNC:** Para establecer transmisión sincrónica o asincrónica
 - 1=Transmisión sincrónica
 - 0=Transmisión asincrónica
- **SENDB:** Para enviar un carácter de break, no lo utilizamos mucho
- **BRGH:** Nos dice cómo vamos a generar el clock, de baja o de alta frecuencia
 - 1= alta frecuencia
 - 0= baja frecuencia
- **TRMT:** Nos dice si el TSR está vacío o lleno
 - 1=vacío
 - 0=lleno
- **TX9D:** dato que vamos a transmitir como noveno bit, en caso de que esté habilitado

¿Qué tengo que hacer para trabajar con el transmisor?

1. El transmisor se habilita al poner: bit **TXEN = 1** en Registro TXSTA
2. Se configura en **modo asíncrono** al poner: bit **SYNC = 0** en Registro TXSTA
3. La transmisión se habilita con **SPEN = 1** en el registro RCSTA y el pin TX/CK se configura automáticamente como salida

Para transmitir un carácter

1. Para iniciar debemos **escribir** un carácter en TXREG
2. Transfiere carácter de TXREG a TSR
3. **TXREG vacío** → **TXIF = 1** en Registro PIR1, interrumpe si TXIE = 1 en Registro PIE1
4. TXIF se pone a 1 pero **no se puede borrar por software**. Se pone a 0 al escribir otro carácter en TXREG
5. Los bits del TSR son desplazados al pin TX sincronizados con reloj (start, b0 ... b7, stop)
6. Cuando **sale el bit stop** del TSR el bit **TRMT** del Registro TXSTA se pone a 1 (vacío)
7. Si TXREG se ha escrito comienza de nuevo el proceso al terminar de transmitir el bit stop

TXIF no va a 0 después de escribir TXREG
 Va a 0 en un tiempo de 2 instrucciones.
 Cuidado al usar polling !!!

COMUNICACIÓN MULTIPUNTO – Maestro Esclavo

Es un artificio para poder transmitir a varios, es decir, tener conectado un chip maestro y varios chips esclavos. Donde sí está habilitado este modo de addres, lo que van a estar haciendo los chips esclavos es estar esperando por su dirección, van a tener configurados una dirección, entonces el maestro va a transmitir una dirección. Una vez que uno de los esclavos detecta su dirección, el próximo dato que venga va a ser un dato para este esclavo.

Nosotros no usamos mucho esto, pero es para comunicar varios chips.

Detección de Dirección:

El bit ADDEN = 1 en el Registro RCSTA

El módulo EUSART recibe sólo caracteres de 9 bits y descarta los de 8 bits

Al recibir cada carácter de 9 bits verifica si los 8 bits de datos son su dirección.

Si detecta su dirección coloca el bit ADDEN = 0 en el Registro RCSTA

Tráfico de Datos:

Una vez establecida la conexión entre dos dispositivos la transmisión/recepción es una comunicación punto a punto con el bit ADDEN = 0 y los 8 bits del carácter son datos

Fin de la Comunicación:

Una vez recibido el último dato se coloca de nuevo el bit ADDEN = 1 y queda disponibles para otro posible inicio de comunicación

**TRANSMISIENDO CON 9 BITS**

- Se coloca TX9 = 1 en el Registro TXSTA
- El bit TX9D del registro TXSTA es el noveno bit y el más significativo.
- Al transmitir un dato de 9 bits, el bit de datos TX9D deberá estar escrito antes de que se escriban los 8 bits menos significativos en el registro TXREG.
- Los nueve bits de datos se copian en registro de desplazamiento TSR inmediatamente después de escribir el carácter en el registro TXREG.

Este modo permite generar el bit de paridad por software

¿Qué es la paridad de un byte?

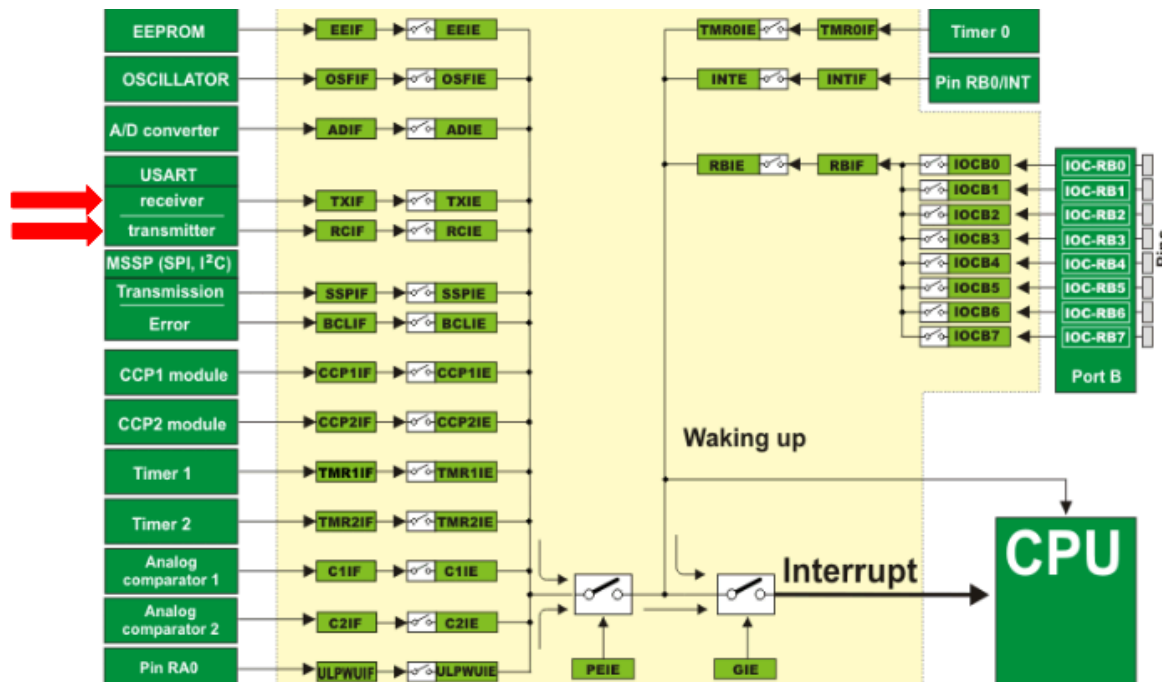
Es la cantidad de 1 que tiene ese byte. Si se trabaja con paridad impar y la cantidad de 1 que contiene mi número es par pone, el bit se setea en '1' y si la cantidad de 1 es impar pone un '0'. De esta manera yo podría detectar si tengo un error de un bit, porque cuando me llegue el dato, si el que yo transmití es par y lo que me llega es un impar -error, debemos pedir retransmisión del dato, ya que no podemos corregirlo nosotros.

REGISTROS ASOCIADOS CON TRANSMISIÓN ASINCRÓNICA

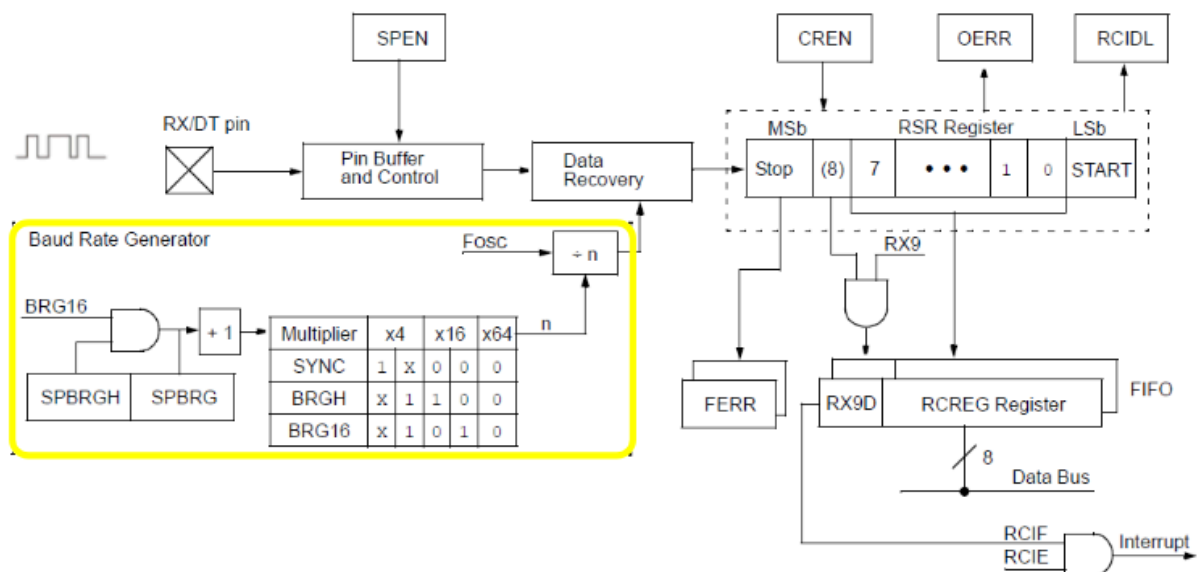
Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR, BOR	Value on all other Resets
BAUDCTL	ABDOVF	RCIDL	—	SCKP	BRG16	—	WUE	ABDEN	01-0 0-00	01-0 0-00
INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000x
PIE1	—	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	-000 0000	-000 0000
PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	-000 0000	-000 0000
RCREG	EUSART Receive Data Register								0000 0000	0000 0000
RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
SPBRG	BRG7	BRG6	BRG5	BRG4	BRG3	BRG2	BRG1	BRG0	0000 0000	0000 0000
SPBRGH	BRG15	BRG14	BRG13	BRG12	BRG11	BRG10	BRG9	BRG8	0000 0000	0000 0000
TRISC	TRISC7	TRISC6	TRISC5	TRISC4	TRISC3	TRISC2	TRISC1	TRISC0	1111 1111	1111 1111
TXREG	EUSART Transmit Data Register								0000 0000	0000 0000
TXSTA	CSRC	TX9	TXEN	SYNC	SENDB	BRGH	TRMT	TX9D	0000 0010	0000 0010

Legend: x = unknown, - = unimplemented read as '0'. Shaded cells are not used for Asynchronous Transmission.

INTERRUPCIONES EN EUSART Módulo Transmisión Serie



EUSART Modo Receptor



Al recibir debo tener un shift register que vaya tomando el dato que va ingresando por el RX/DT pin. **Buffer control** lo que hace es decirle toma todo el circuito de lectura de datos. Pero, además de eso necesitamos un circuito que nos permita recuperar los datos, porque cuando uno manda los datos por el canal no está muy bonito como se aprecia en la imagen. Entonces, el circuito debería samplear donde más fuerza tenga el dato y para eso existe el circuito de recuperación de datos **Data Recovery**, la idea es que tome el dato en el mejor momento posible. Este circuito debe tener un clock también. A la salida del DR tenemos una imagen más perfecta de '0' y '1'. Una vez que recupero el bit, recién ahí lo mando a un shift register. **Una vez que llega un dato completo se genera una interrupción.** En este caso tenemos un buffer de dos registros, donde se recibe un dato, el cual va a caer en la primera posición, luego si llega un segundo dato éste va a pasar a la segunda posición. Si llega un tercer dato, éste nos va a sobrescribir la primera posición si yo no la leí. Ahí es cuando se produce un error de **overwrite**, debido a que me siguieron entrando datos y hubo uno que no leí, me lo perdí.

El **FERR** nos detecta un **error por STOP**, porque no detectó bien el bit de STOP, cuando tenía que llegar un '1' llegó un '0'. (BIT DE STOP es un 1 y el BIT DE START es un 0). Si una vez que empezó a detectar datos, cuando llega el tiempo del bit de STOP y en lugar de llegar un '1' llegó un '0' se genera un **FERR**

RCSTA: Registro de Estado y Control de Recepción

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R-0	R-0	R-x
SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D
bit 7							bit 0

Sincrónico

Sinc/Asinc

bit 7	SPEN: Serial Port Enable bit 1 = Serial port enabled (configures RX/DT and TX/CK pins as serial port pins) 0 = Serial port disabled (held in Reset)
bit 6	RX9: 9-bit Receive Enable bit 1 = Selects 9-bit reception 0 = Selects 8-bit reception
bit 5	SREN: Single Receive Enable bit <u>Asynchronous mode:</u> Don't care <u>Synchronous mode – Master:</u> 1 = Enables single receive 0 = Disables single receive This bit is cleared after reception is complete. <u>Synchronous mode – Slave</u> Don't care
bit 4	CREN: Continuous Receive Enable bit <u>Asynchronous mode:</u> 1 = Enables receiver 0 = Disables receiver <u>Synchronous mode:</u> 1 = Enables continuous receive until enable bit CREN is cleared (CREN overrides SREN) 0 = Disables continuous receive

Habilitación para recibir

- El receptor se habilita al poner: **bit CREN = 1** en Registro RCSTA
- Se configura en modo asíncrono al poner: **bit SYNC = 0** en Registro TXSTA
- Se habilita la recepción con **SPEN = 1** en el registro RCSTA y el pin RX/DT se configura automáticamente como entrada

Recibiendo un carácter

- Al detectar el flanco descendente de un bit start el carácter se va cargando en RSR
- Al recibir el bit stop el carácter se transfiere al Registro RCREG (si está vacío)
- Se pone el bit **CRIF = 1** en Registro PIR1, interrumpe si RCIE = 1 en Registro PIE1.
- CRIF se pone a 1 pero no se puede borrar por software. Se pone a 0 al leer el byte de RCREG (RCREG es un buffer de dos bytes tipo FIFO)
- Para habilitar la recepción de un dato de 9 bits se pone a uno el bit RX9 del registro RCSTA

ERRORES EN RECEPCIÓN

Errores de sobreescritura

- Si se detecta un nuevo START y el RCREG está lleno (tiene 2 bytes) se pone en 1 el flag ORR (Registro RCSTA).
- Se pierde un 3er. byte que esté ingresando por RX
- Se debe resetear la EUSART:
 1. Colocando un 0 y a posteriori 1 en el bit SPEN o
 2. poniendo un 0 y a posteriori 1 en el bit CREN, ambos del Registro RCSTA

Error de Frame:

- Un carácter termina con STOP. Si no se detecta el STOP esperado (1 lógico) se levanta el flag FERR.
- Cada carácter del buffer FIFO tiene su propio bit de error FERR. Si FERR = 1 en el Registro RCSTA hubo error en el primer dato de la FIFO
- Al leer del RCREG el primer carácter se borra FERR y queda el del segundo carácter
- Al leer del RCREG el segundo carácter se borra FERR del segundo carácter • El bit FERR puede ser forzado a 0 poniendo SPEN = 0

RECIBIENDO CON 9 BITS

- Se prepara para 9 bits colocando **RX9 = 1** en el Registro RCSTA
- El noveno bit se escribe automáticamente en el **bit RX9D** del Registro RCSTA
- **Se debe leer primero el bit RX9D y luego el RCREG.** Esta secuencia es obligatoria porque al leer RCREG el bit RX9D se pone a 0.

Si este modo es usado para verificar paridad debe analizarse si hay errores por software

REGISTROS ASOCIADOS CON TRANSMISIÓN ASINCRÓNICA

	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR, BOR	Value on all other Resets
	BAUDCTL	ABDOVF	RCIDL	—	SCKP	BRG16	—	WUE	ABDEN	01-0 0-00	01-0 0-00
	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000x
INT	PIE1	—	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	-000 0000	-000 0000
	PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	-000 0000	-000 0000
	RCREG	EUSART Receive Data Register								0000 0000	0000 0000
	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
	SPBRG	BRG7	BRG6	BRG5	BRG4	BRG3	BRG2	BRG1	BRG0	0000 0000	0000 0000
	SPBRGH	BRG15	BRG14	BRG13	BRG12	BRG11	BRG10	BRG9	BRG8	0000 0000	0000 0000
	TRISC	TRISC7	TRISC6	TRISC5	TRISC4	TRISC3	TRISC2	TRISC1	TRISC0	1111 1111	1111 1111
	TXREG	EUSART Transmit Data Register								0000 0000	0000 0000
	TXSTA	CSRC	TX9	TXEN	SYNC	SENDER	BRGH	TRMT	TX9D	0000 0010	0000 0010

Legend: x = unknown, — = unimplemented read as '0'. Shaded cells are not used for Asynchronous Reception.

GENERADOR DE BAUDIOS

- Temporizador dedicado de 8 o 16 bits
- Se usa tanto para sincrónico como asincrónico
- Para un Fosc del sistema se debe elegir la combinación que de un error menor
- Es un temporizador por como el TMR0

REGISTROS GENERADORES DE BAUDIOS: SPBRGH:SPBRG

BRG15	BRG14	BRG13	BRG12	BRG11	BRG10	BRG9	BRG8	BRG7	BRG6	BRG5	BRG4	BRG3	BRG2	BRG1	BRG0
-------	-------	-------	-------	-------	-------	------	------	------	------	------	------	------	------	------	------

FÓRMULAS PARA EL BAUD RATE

Configuration Bits			BRG/EUSART Mode	Baud Rate Formula
SYNC	BRG16	BRGH		
0	0	0	8-bit/Asynchronous	$F_{osc}/[64 (n+1)]$
0	0	1	8-bit/Asynchronous	$F_{osc}/[16 (n+1)]$
0	1	0	16-bit/Asynchronous	
0	1	1	16-bit/Asynchronous	$F_{osc}/[4 (n+1)]$
1	0	x	8-bit/Synchronous	
1	1	x	16-bit/Synchronous	

Legend: x = Don't care, n = value of SPBRGH, SPBRG register pair

FÓRMULAS DE VALORES DE VELOCIDAD, TEMPORIZADOR Y ERROR.

$$\text{Velocidad de transmisión en baudios deseada} = \frac{F_{osc}}{64(SPBRGH:SPBRG - 1)}$$

Donde (SPBRGH:SPBRG) sería lo que yo venga cargado en esos registros como un solo número de 16 bits

$$SPBRGH:SPBRG = \frac{\frac{F_{osc}}{\text{Velocidad de transmisión en baudios deseada}} - 1}{64}$$

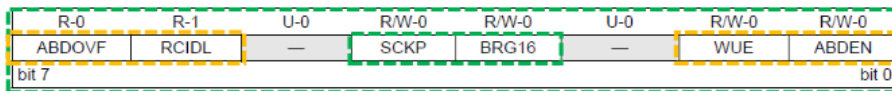
Si yo quisiera saber cual es el valor que tengo que cargar esos registros, utilizo esta fórmula para calcular en base a la velocidad de transmisión que yo quiero. Lo más probable es que el resultado no sea un número entero, yo voy a tener que elegir entre ir hacia arriba o hacia abajo en ese número fraccional.

$$\text{Error [\%]} = \frac{\text{Velocidad de transmisión en baudios calculada} - \text{Velocidad de transmisión en baudios deseada}}{\text{Velocidad de transmisión en baudios deseada}}$$

Al elegir el número anterior voy a generar un error que va a estar dado por esta fórmula. Es muy importante tratar de llevar este error a lo mínimo posible.

Estas fórmulas ya no se utilizan ya que los valores se encuentran tabulados

BAUDCTL: Registro de Control Velocidad de Transmisión



bit 7 **ABDON**: Auto-Baud Detect Overflow bit

Asynchronous mode:

1 = Auto-baud timer overflowed

0 = Auto-baud timer did not overflow

Synchronous mode:

Don't care

bit 6 **RCIDL**: Receive Idle Flag bit

Asynchronous mode:

1 = Receiver is Idle

0 = Start bit has been received and the receiver is receiving

Synchronous mode:

Don't care

bit 5 **Unimplemented**: Read as '0'

bit 4 **SCKP**: Synchronous Clock Polarity Select bit

Asynchronous mode:

1 = Transmit inverted data to the RB7/TX/CK pin

0 = Transmit non-inverted data to the RB7/TX/CK pin

Synchronous mode:

1 = Data is clocked on rising edge of the clock

0 = Data is clocked on falling edge of the clock

bit 3 **BRG16**: 16-bit Baud Rate Generator bit

1 = 16-bit Baud Rate Generator is used

0 = 8-bit Baud Rate Generator is used

bit 2 **Unimplemented**: Read as '0'

bit 1 **WUE**: Wake-up Enable bit

Asynchronous mode:

1 = Receiver is waiting for a falling edge. No character will be received byte RCIF will be set. WUE will automatically clear after RCIF is set.

0 = Receiver is operating normally

Synchronous mode:

Don't care

bit 0 **ABDEN**: Auto-Baud Detect Enable bit

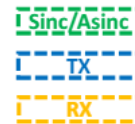
Asynchronous mode:

1 = Auto-Baud Detect mode is enabled (clears when auto-baud is complete)

0 = Auto-Baud Detect mode is disabled

Synchronous mode:

Don't care



REGISTROS ASOCIADOS CON GENERACIÓN DE BAUDIOS

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR, BOR	Value on all other Resets
BAUDCTL	ABDON	RCIDL	—	SCKP	BRG16	—	WUE	ABDEN	01-0 0-00	01-0 0-00
RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
SPBRG	BRG7	BRG6	BRG5	BRG4	BRG3	BRG2	BRG1	BRG0	0000 0000	0000 0000
SPBRGH	BRG15	BRG14	BRG13	BRG12	BRG11	BRG10	BRG9	BRG8	0000 0000	0000 0000
TXSTA	CSRC	TX9	TXEN	SYNC	SENDER	BRGH	TRMT	TX9D	0000 0010	0000 0010

Legend: x = unknown, — = unimplemented read as '0'. Shaded cells are not used for the Baud Rate Generator.

Transmisión de datos a través de la comunicación EUSART asincrónica:

La velocidad de transmisión deseada debe ajustarse con el uso de los bits BRGH (registro TXSTA), BRG16 (registro BAUDCTL) y registros SPBRGH y SPBRG. Si yo no seteo la velocidad nunca se van a poder comunicar los dos bien, el PIC con el otro PIC, o el PIC con la computadora.

El bit SYNC (registro TXSTA) debe ser cero y el bit SPEN se debe ser uno (registro RCSTA) con el fin de **habilitar el puerto serie y de modo asincrónico**

En la transmisión de datos de 9 bits, el bit TX9 del registro TXSTA debe ser uno

La transmisión de datos se habilita poniendo a uno el bit TXEN del registro TXSTA. El bit TXIF del registro PIR1 se ajusta automáticamente

Para que el bit TXEN provoque una interrupción, los bits TXIE del registro PIE1 y los bits GIE y PEIE del registro INTCON deben estar en 1

~~En la transmisión de datos de 9 bits, el valor del noveno bit se debe escribir en el bit TX9D del registro TXSTA~~

La transmisión comienza escribiendo un dato de 8 bits en el registro TXREG.

Recepción de datos a través de la comunicación EUSART asincrónica:

La velocidad en baudios debe seleccionarse mediante el uso de los bits BRGH (registro TXSTA), BRG16 (registro BAUDCTL) y los registros SPBRGH y SPBRG

El bit SYNC (registro TXSTA) debe ser cero y el bit SPEN se debe poner a uno (registro RCSTA) con el fin de **habilitar el puerto serie**

Si es necesario que los datos recibidos generen una interrupción, tanto el bit RCIE del registro PIE1 y los bits GIE y PEIE del registro INTCON deben colocarse en uno

Para la recepción de datos de 9 bits, el bit RX9 del registro RCSTA debe colocarse en uno

La recepción de datos es habilitada poniendo en uno el bit CREN del registro RCSTA

El registro RCSTA debe leerse para obtener información sobre los posibles errores que se hayan producido durante la recepción. El valor del noveno bits será almacenado en este ~~registro si se está trabajando con 9 bits~~

El dato de 8 bits será almacenado en el registro RCREG de dónde se lo deberá leer para su procesamiento en el software